

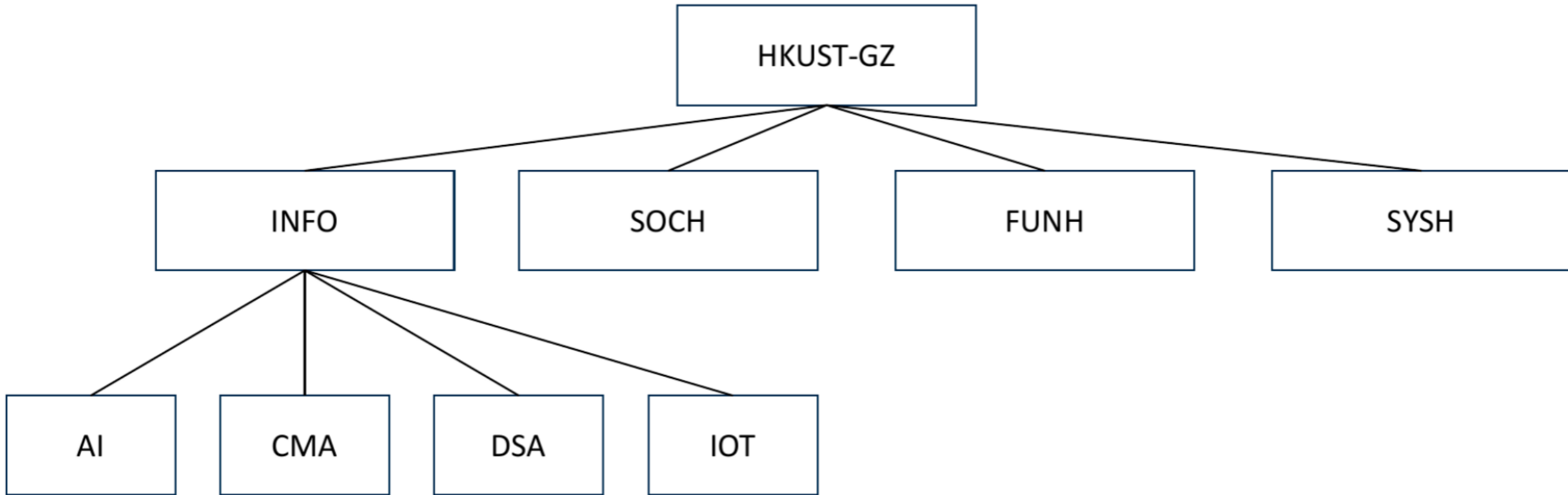
Algorithms on Trees

- Tree Terminology
- Tree Traversal
- Tree DP

Yanlin Zhang | DSAA 2043 Spring 2026

Tree Terminology

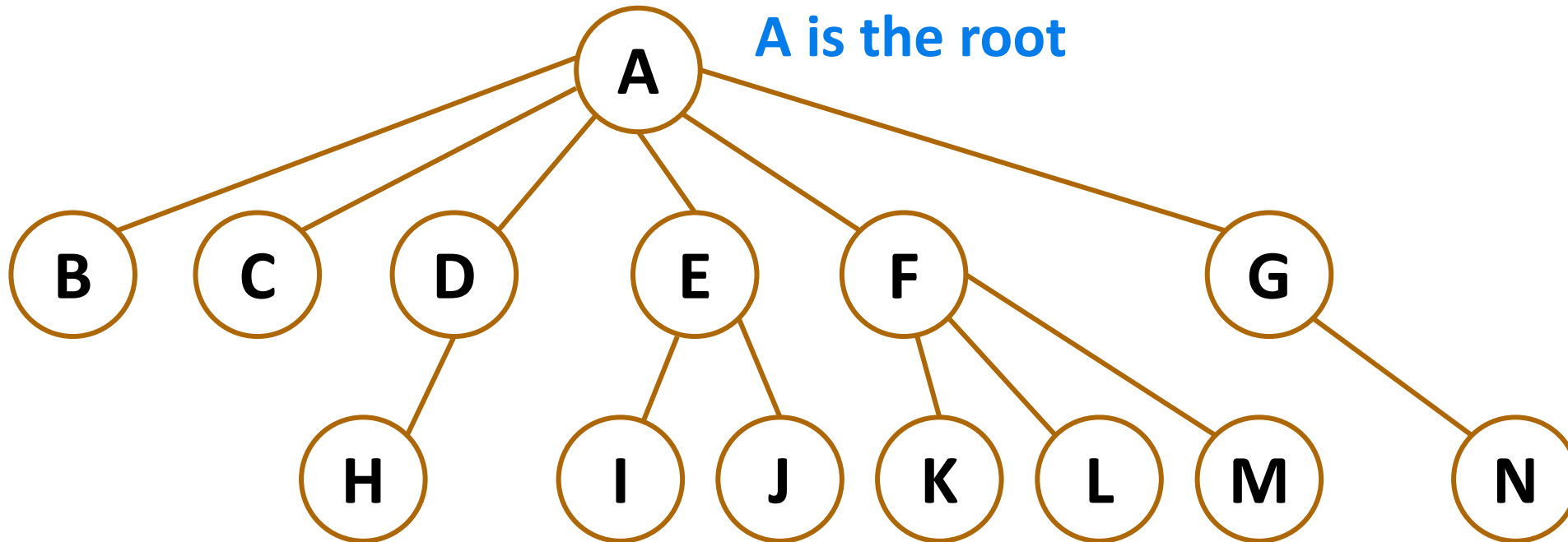
Trees Can Describe Hierarchies



Trees are inherently recursive

What is a Tree (in Computer Science)?

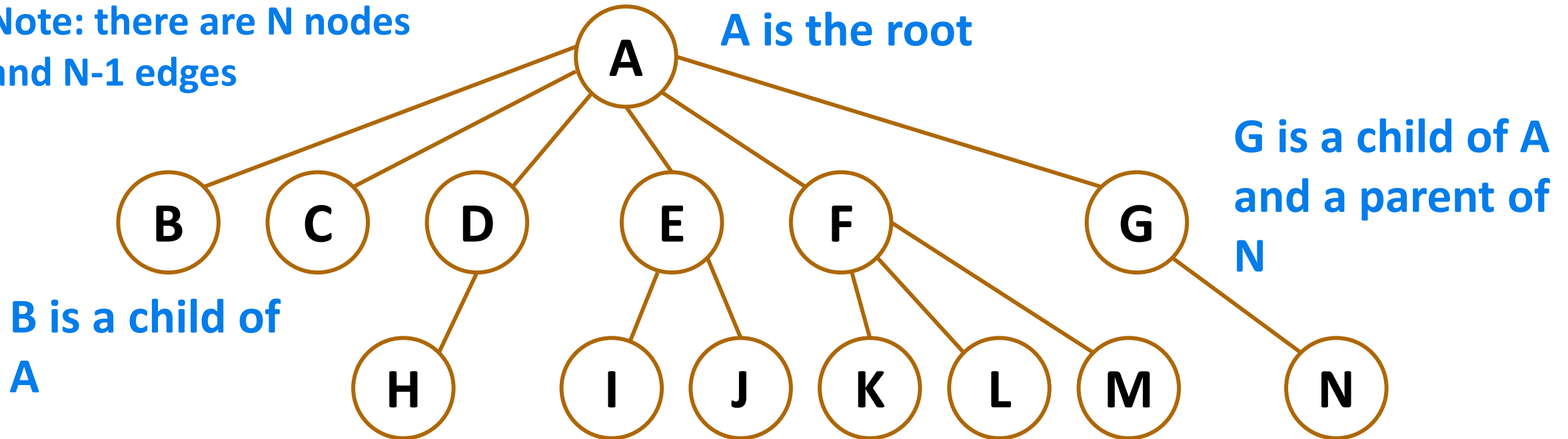
- A tree is a collection of **nodes**, which can be empty. If it is not empty, there is a “root” node, r , and **zero or more non-empty subtrees**, $T_1, T_2, \dots, T_k$, whose roots are connected by a directed edge from r .



What is a Tree (in Computer Science)?

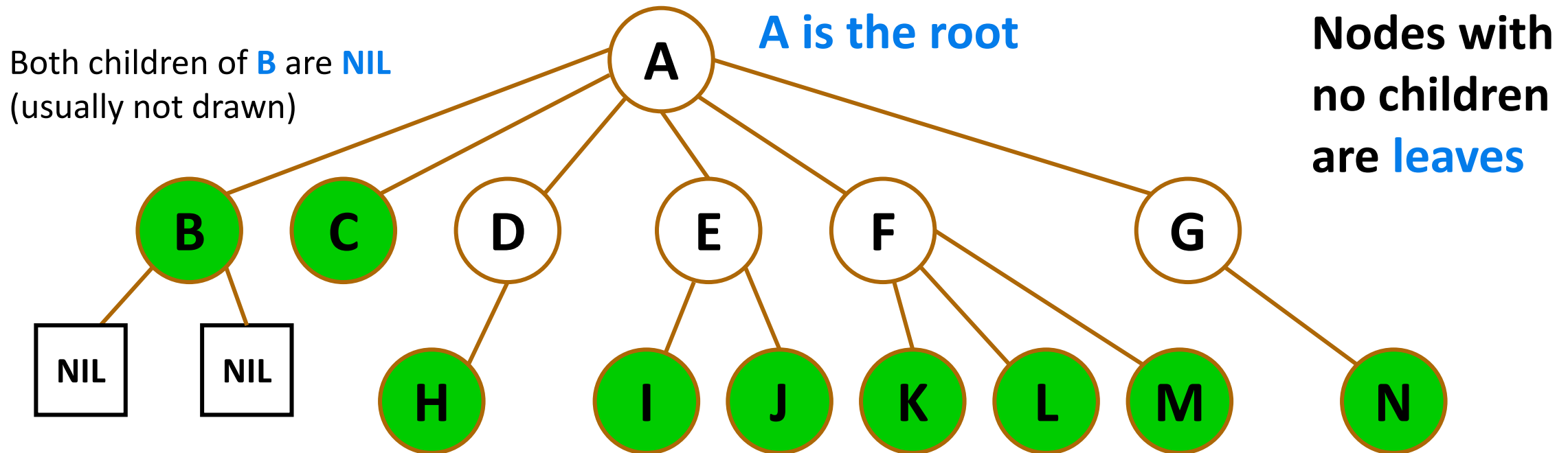
- A tree is a collection of **nodes**, which can be empty. If it is not empty, there is a “root” node, r , and **zero or more non-empty subtrees**, $T_1, T_2, \dots, T_k$, whose roots are connected by a directed edge from r .

Note: there are N nodes
and $N-1$ edges



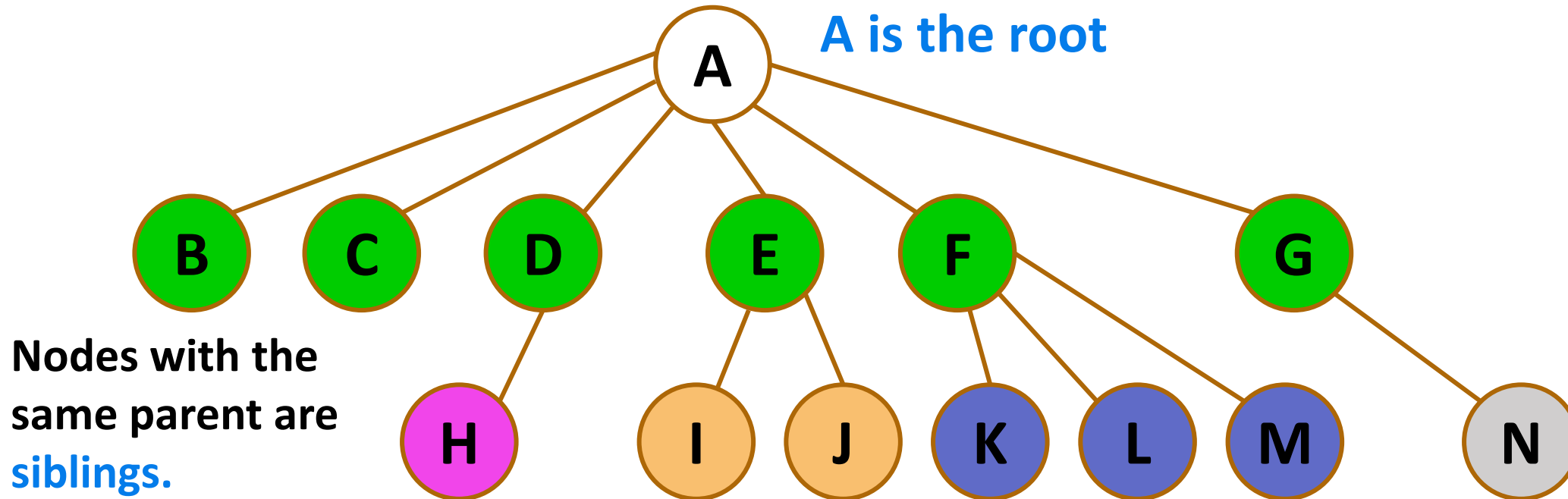
What is a Tree (in Computer Science)?

- A tree is a collection of **nodes**, which can be empty. If it is not empty, there is a “root” node, r , and **zero or more non-empty subtrees**, $T_1, T_2, \dots, T_k$, whose roots are connected by a directed edge from r .



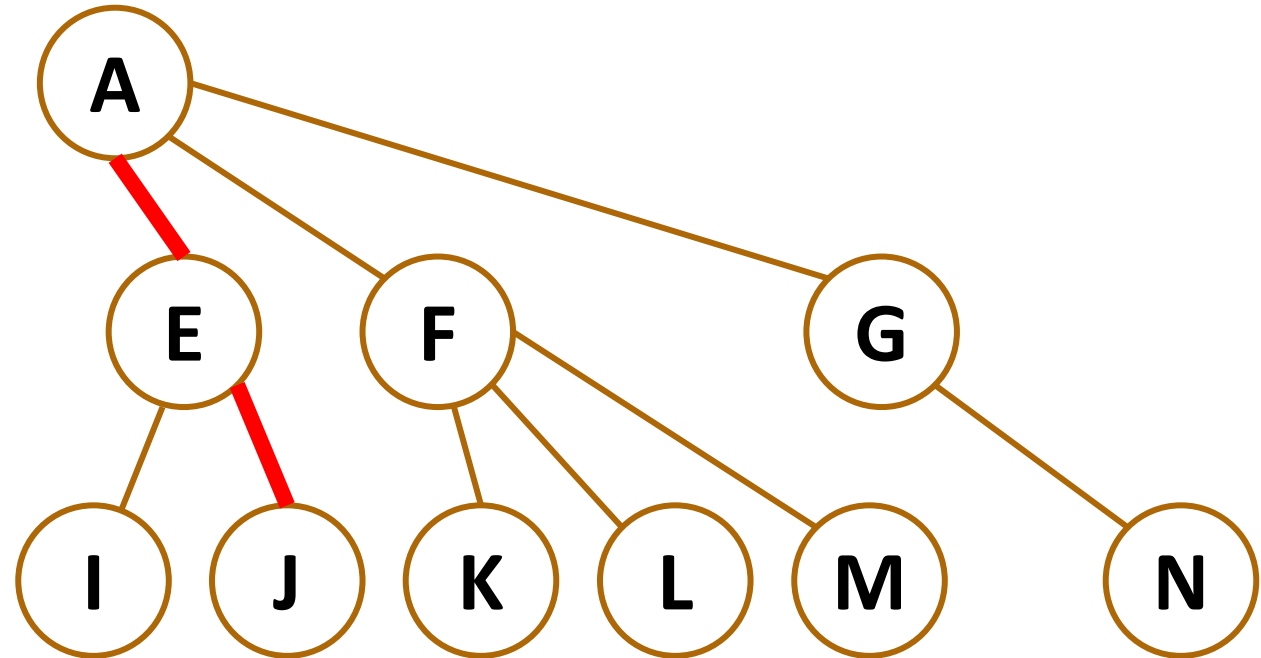
What is a Tree (in Computer Science)?

- A tree is a collection of **nodes**, which can be empty. If it is not empty, there is a “root” node, r , and **zero or more non-empty subtrees**, $T_1, T_2, \dots, T_k$, whose roots are connected by a directed edge from r .



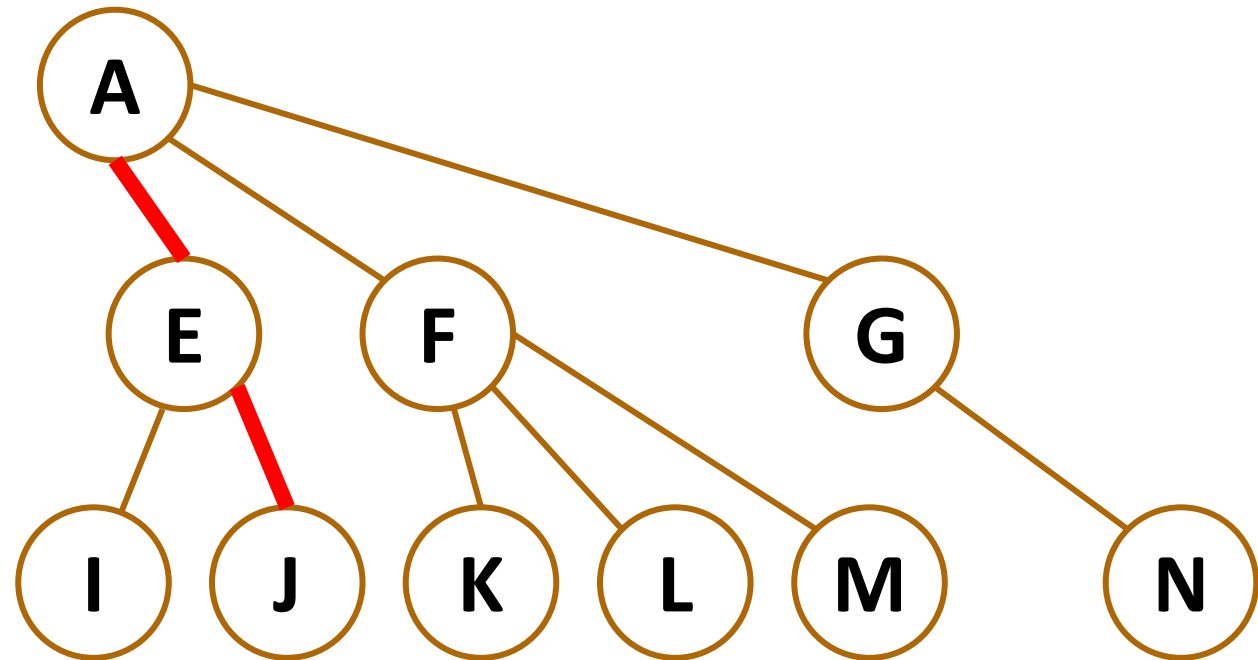
We can define a **path** from a parent to its children.

The path A-E-J has a **length** of two (the number of edges)

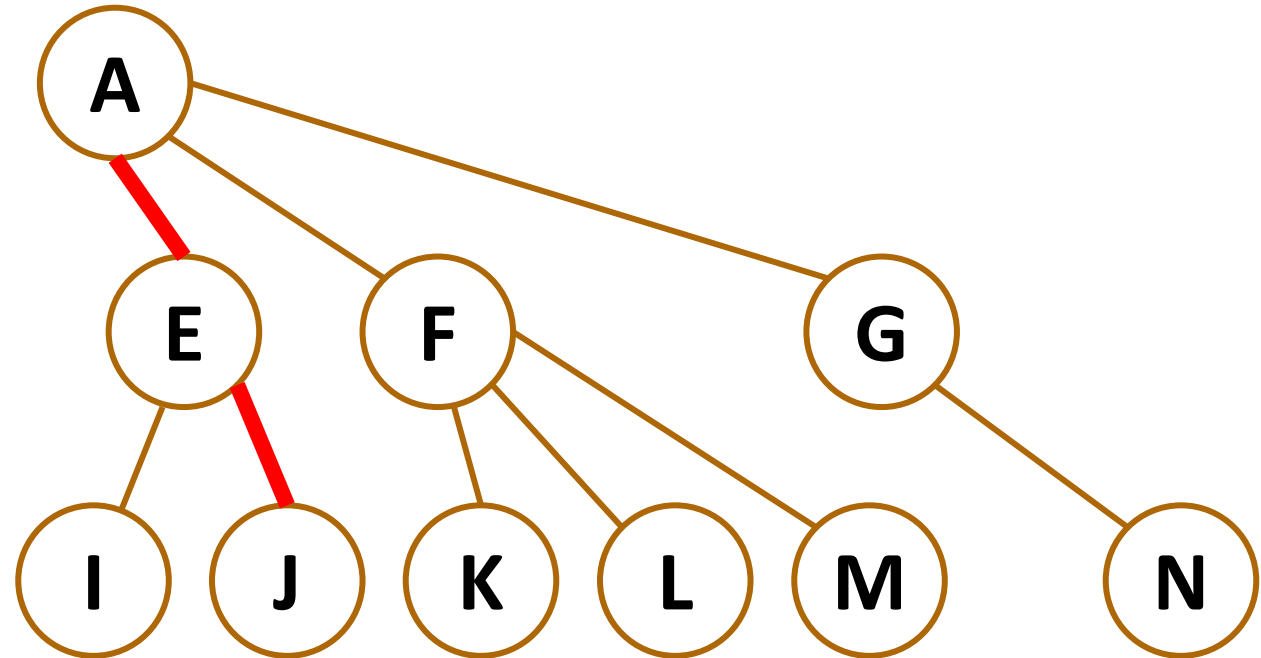


The **depth** of a node is the length from the root. The depth of node J is 2. The depth of the root is 0.

The **height** of a node is the longest path from the node to a leaf. The height of node F is 1. The height of all leaves is 0.

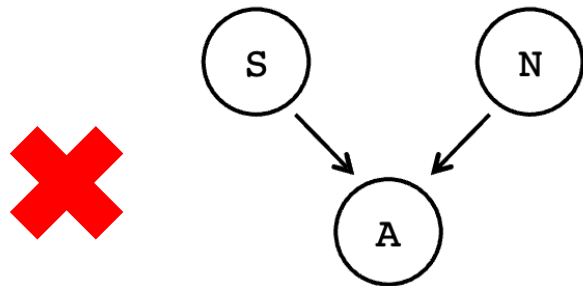


The **height** of a tree is the height of the root (in this case, the height of the tree is 2).

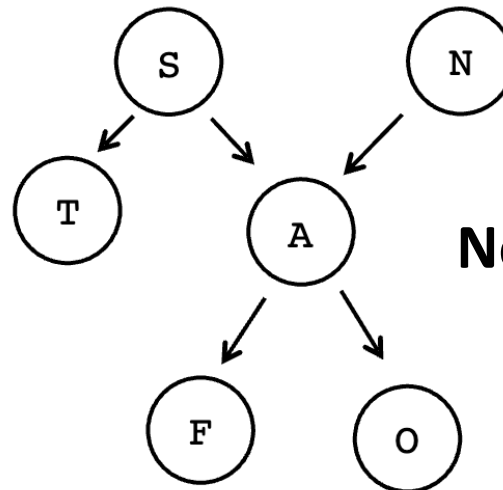


Tree Terminology

Trees can have **only one** parent, and **cannot** have cycles



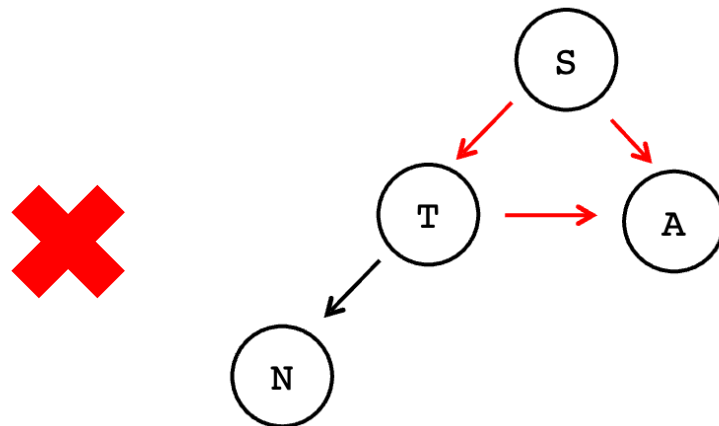
Node A has two parents



Node A has two parents

Tree Terminology

Trees can have **only one** parent, and **cannot** have cycles



not a tree: the red edges make a cycle

Binary Tree Terminology

A tree is a binary tree so that

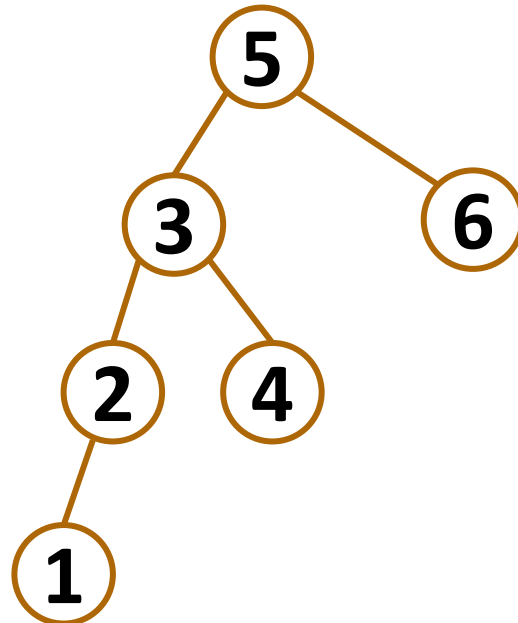
- Each node has at most two children

A **Binary Search Tree (BST)** is a binary tree so that

- Every LEFT descendant of a node has key less than that node.
- Every RIGHT descendant of a node has key larger than that node.

**2 is a descendant
of 5.**

**5 is an ancestor of
2.**



Traversal

Iterate through data structure

Iterate: process elements of data structure

- Sum all elements
- Print each element
- ...

Array

2	1	3	0
---	---	---	---

Forwards: 2, 1, 3, 0

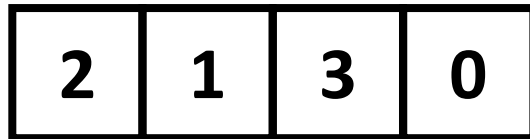
Backwards: 0, 3, 1, 2

Iterate through data structure

Iterate: process elements of data structure

- Sum all elements
- Print each element
- ...

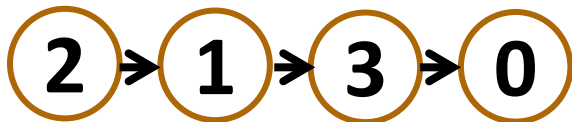
Array



Forwards: 2, 1, 3, 0

Backwards: 0, 3, 1, 2

Linked List



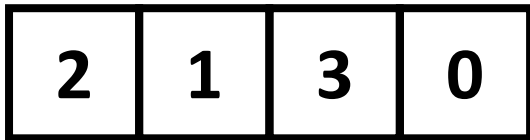
Forwards: 2, 1, 3, 0

Iterate through data structure

Iterate: process elements of data structure

- Sum all elements
- Print each element
- ...

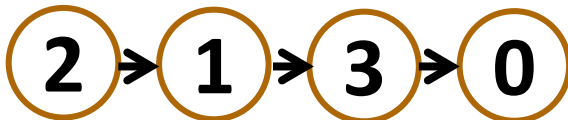
Array



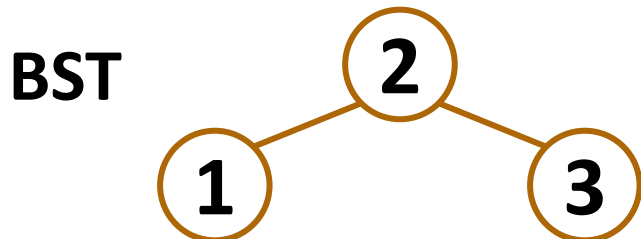
Forwards: 2, 1, 3, 0

Backwards: 0, 3, 1, 2

Linked List

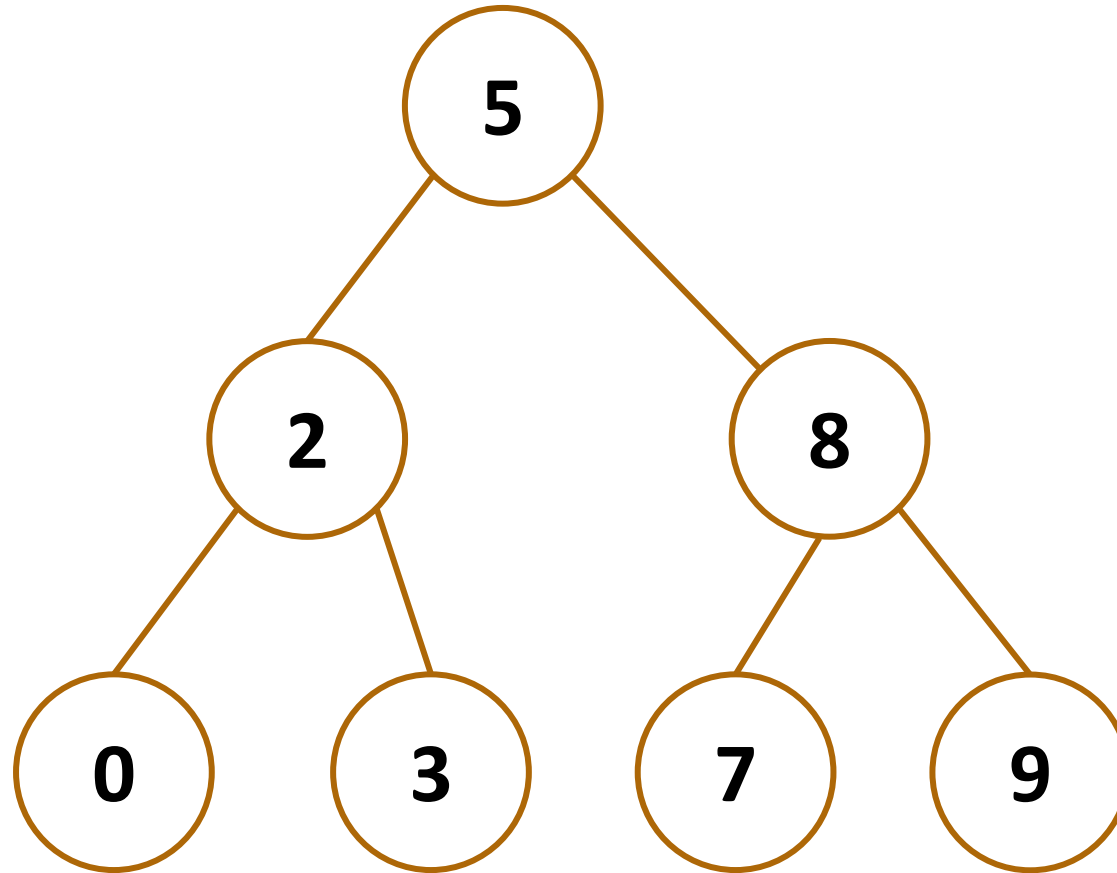


Forwards: 2, 1, 3, 0



???

Iterate through data structure



Discuss: What would a reasonable order be?

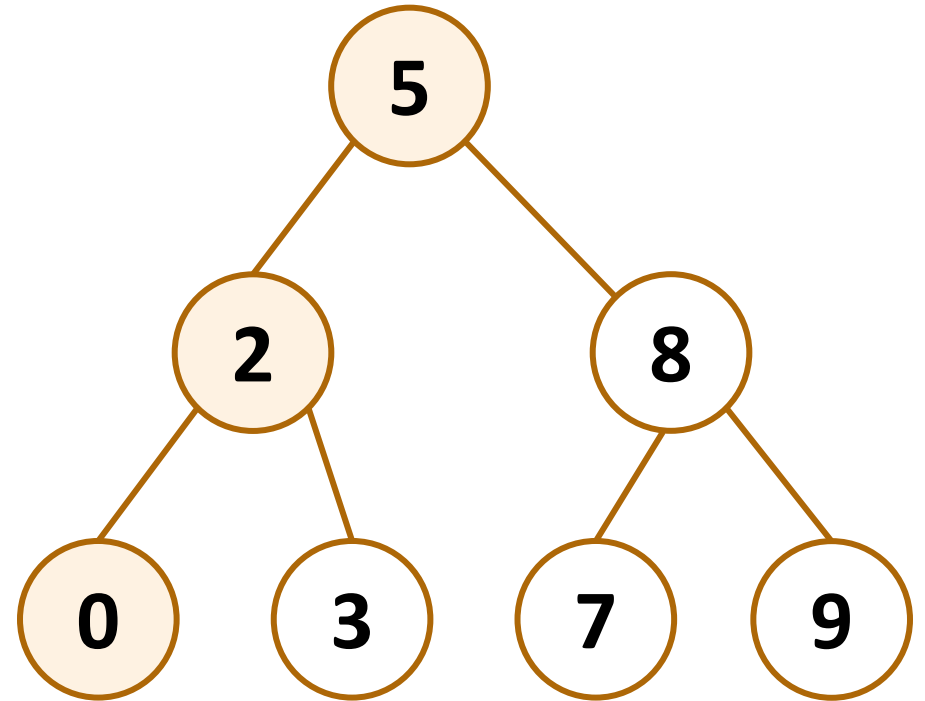
Iterating through tree is **tree traversal : A systematic way of accessing (or visiting) all the nodes of a tree**

Well-known recursive tree traversal algorithms:

- **Depth First Search (DFS)**
 - ✓ Preorder
 - ✓ Inorder
 - ✓ Postorder
- **Breadth First Search (BFS)**
 - ✓ Level order

DFS: go deep, then backtrack

- Start at the root.
- Keep following one child until you reach a leaf.
- Then backtrack to the most recent node with an unvisited child.

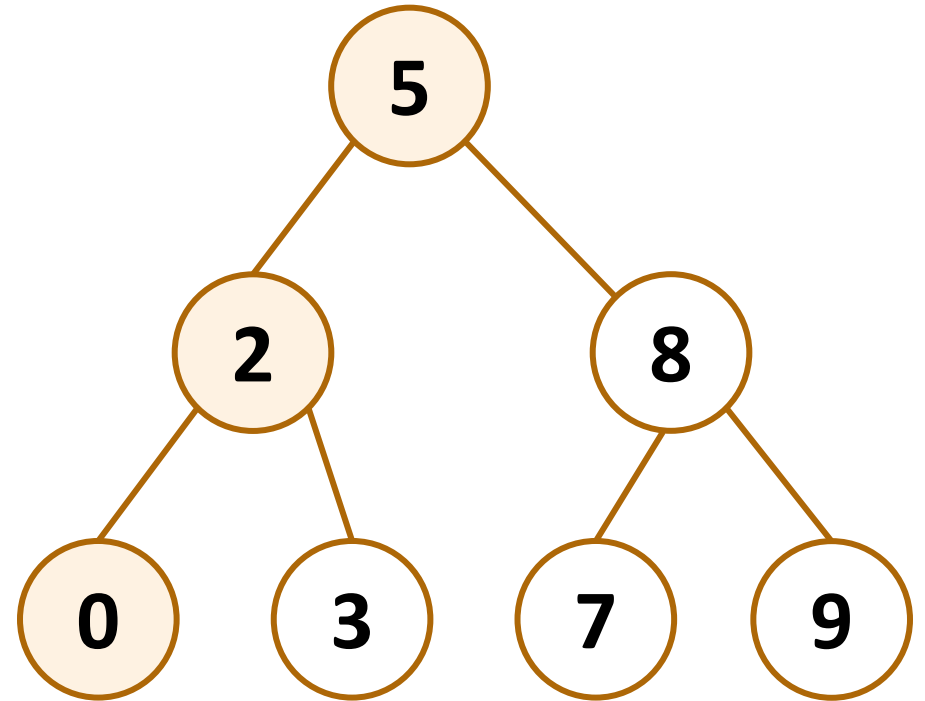


DFS: go deep, then backtrack

Recursive idea

```
1 def dfs(u):  
2     visit(u)  
3     for v in children[u]:  
4         dfs(v)
```

visit u first
then recursively process every child

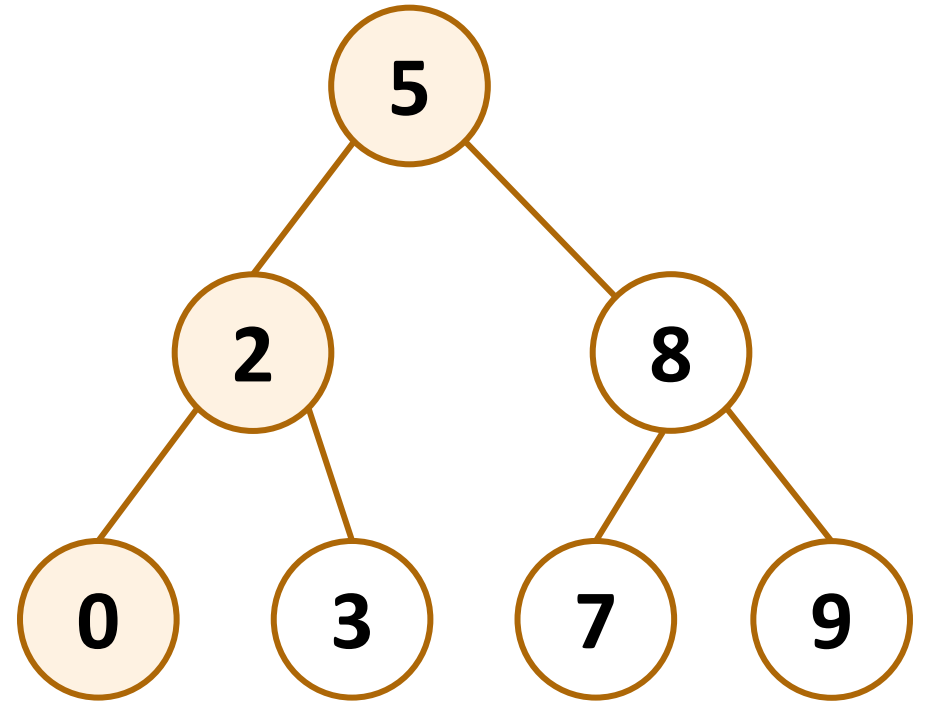


DFS walkthrough

Visited so far



We reached a leaf, so the next action is backtracking.

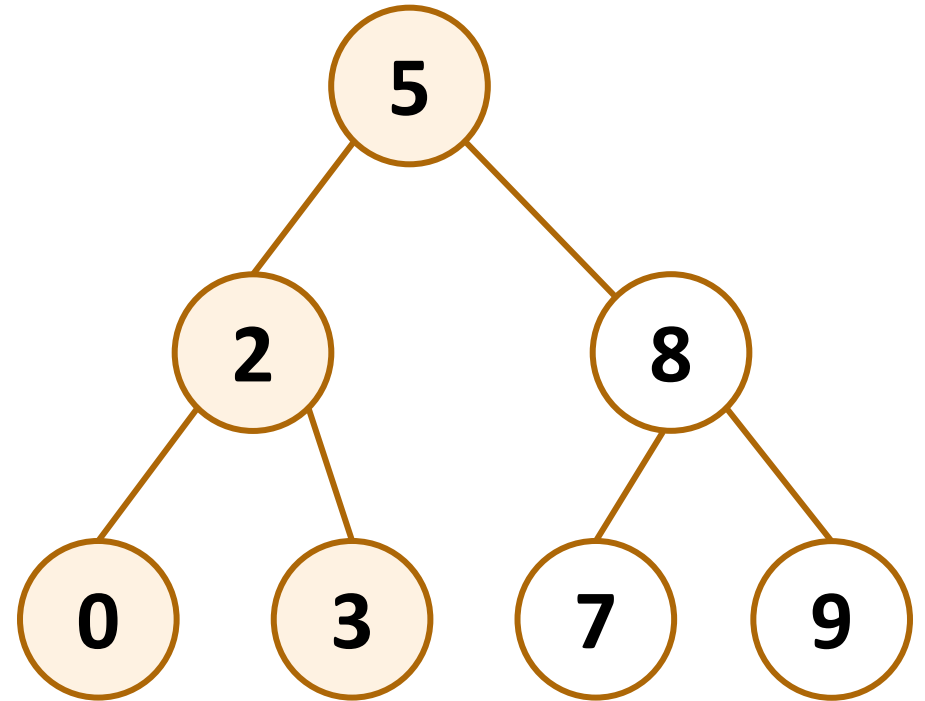


DFS walkthrough

Visited so far

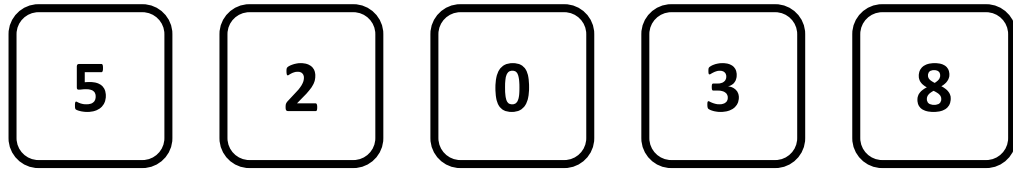


Backtrack to **2** and then continue with its next child **3**.

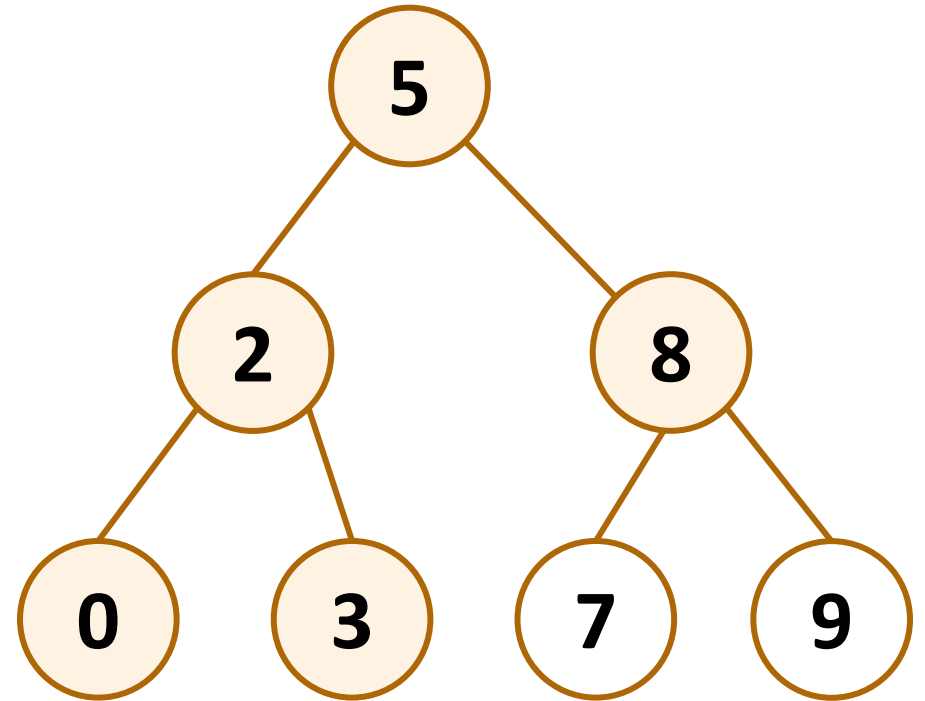


DFS walkthrough

Visited so far

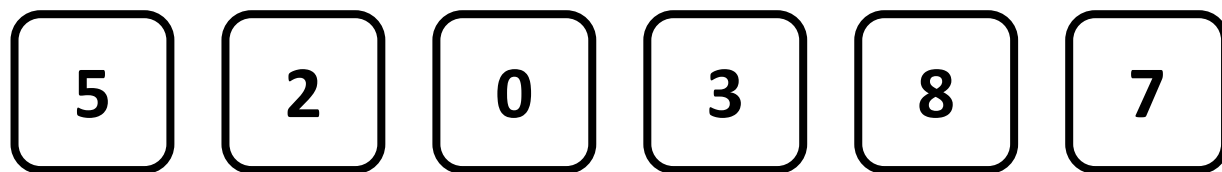


Backtrack to **5** and then continue with its next child **8**.

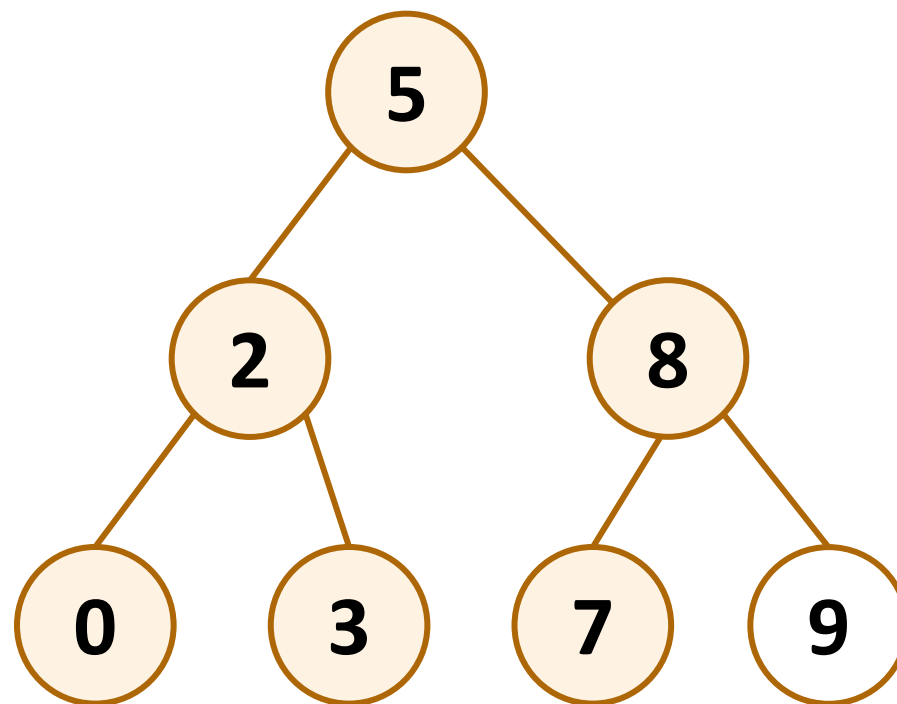


DFS walkthrough

Visited so far



Go to **7**.

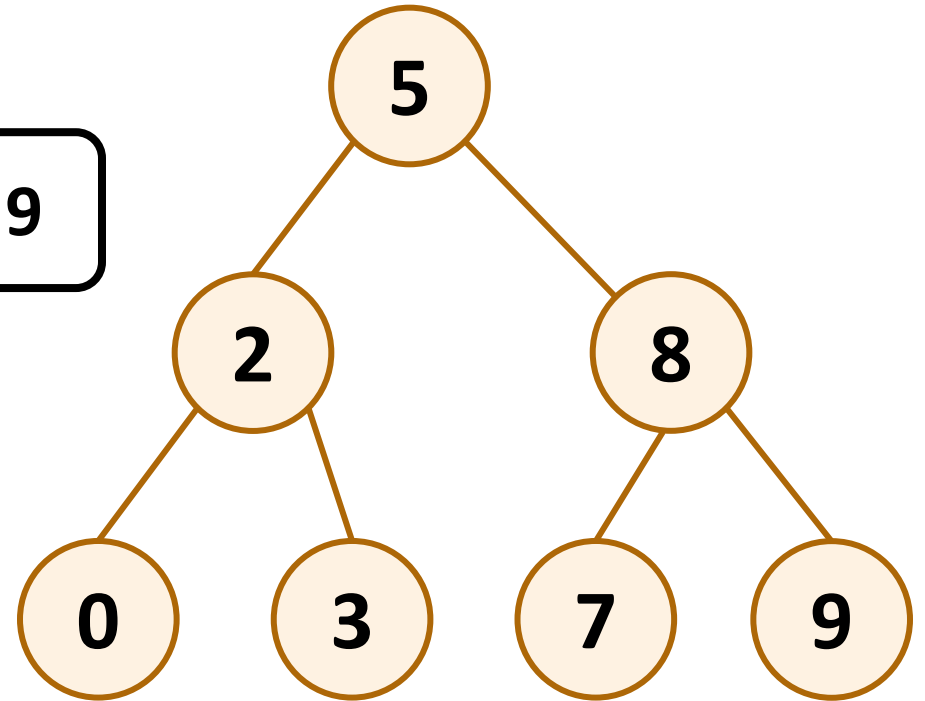


DFS walkthrough

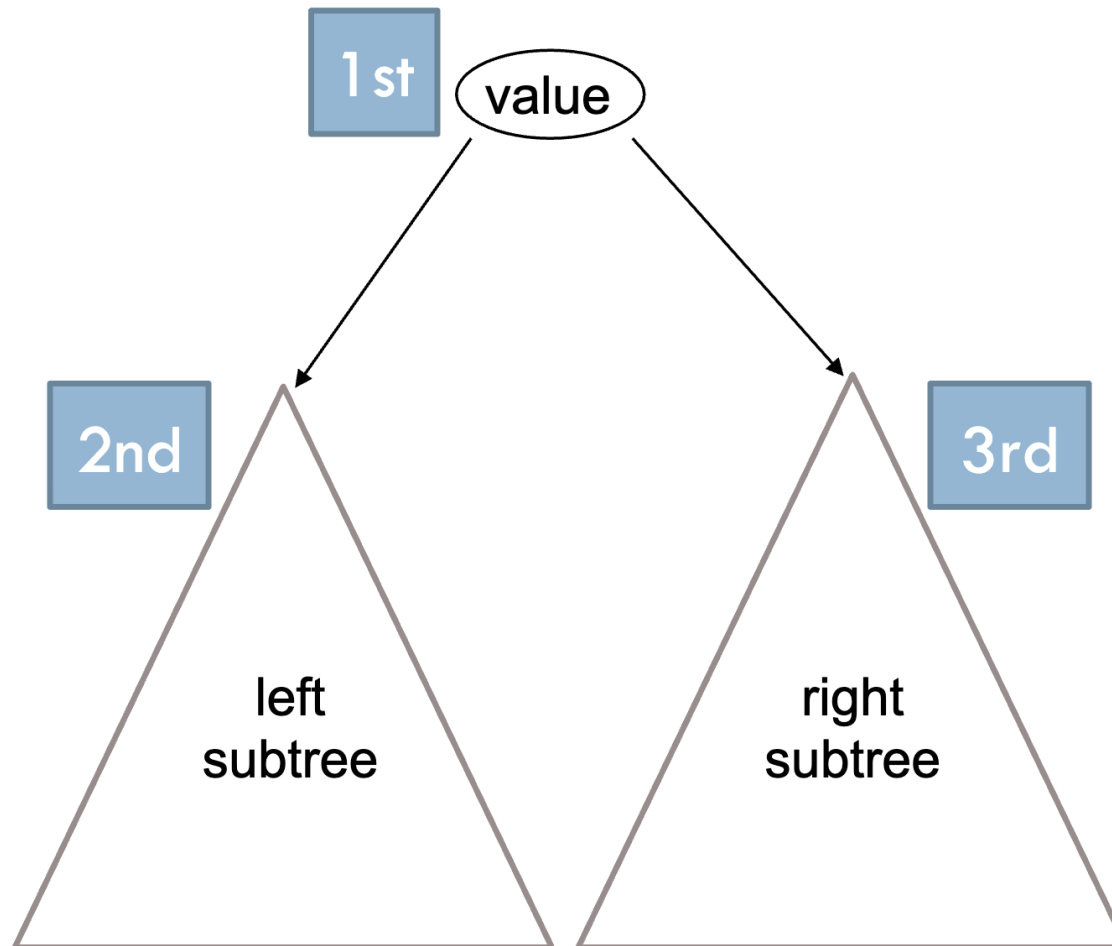
Visited so far



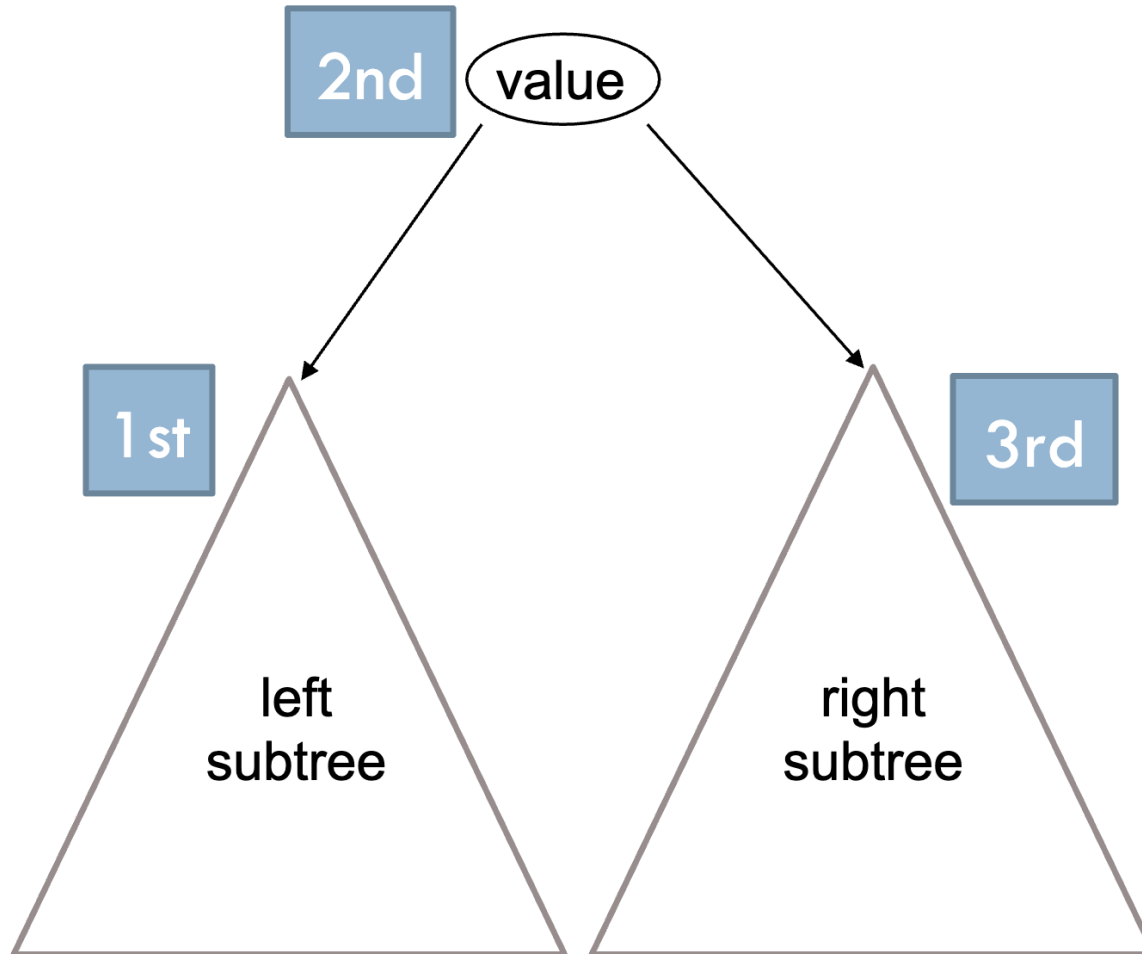
Backtrack to **8** and then continue with its next child **9**.



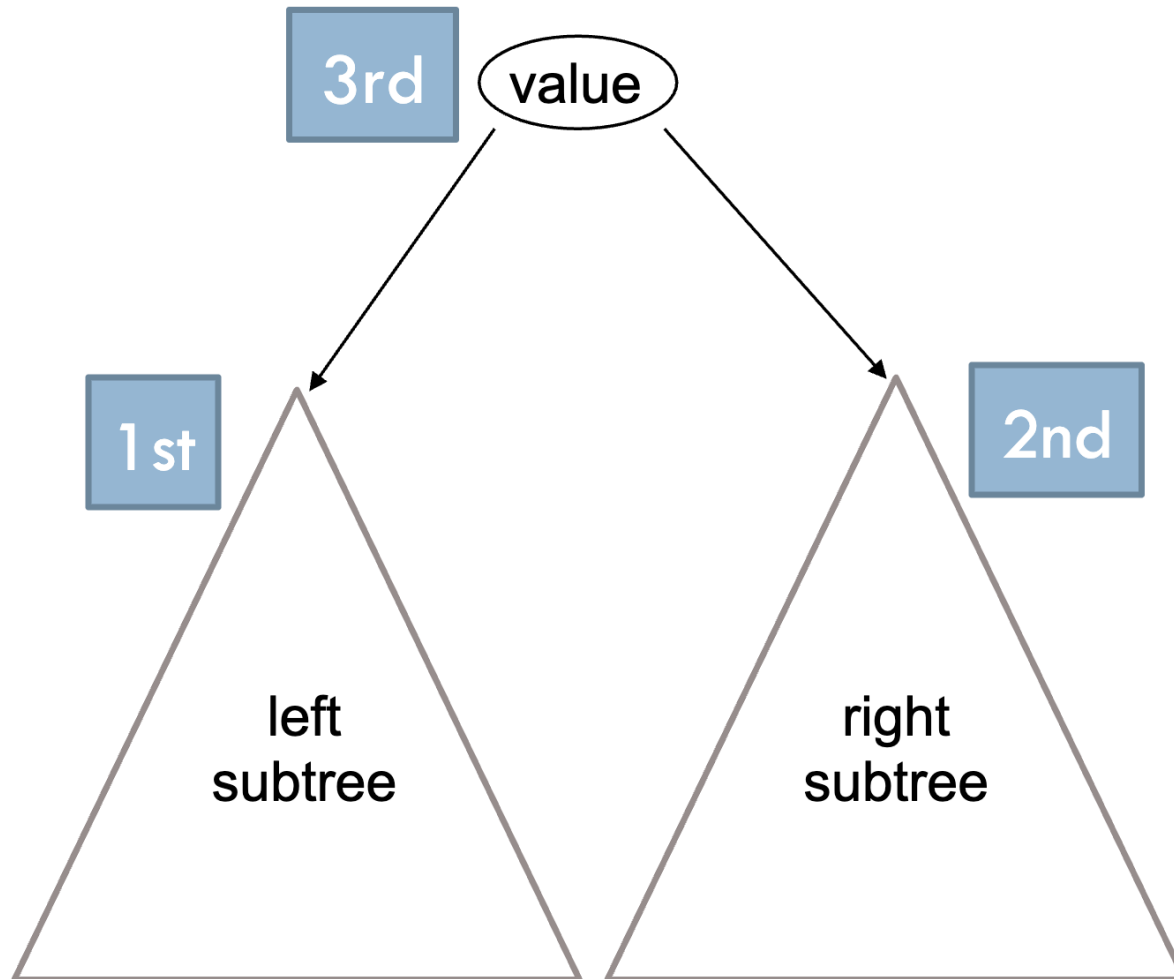
Pre: process root before subtrees



In: process root in-between subtrees



Post: process root after subtrees

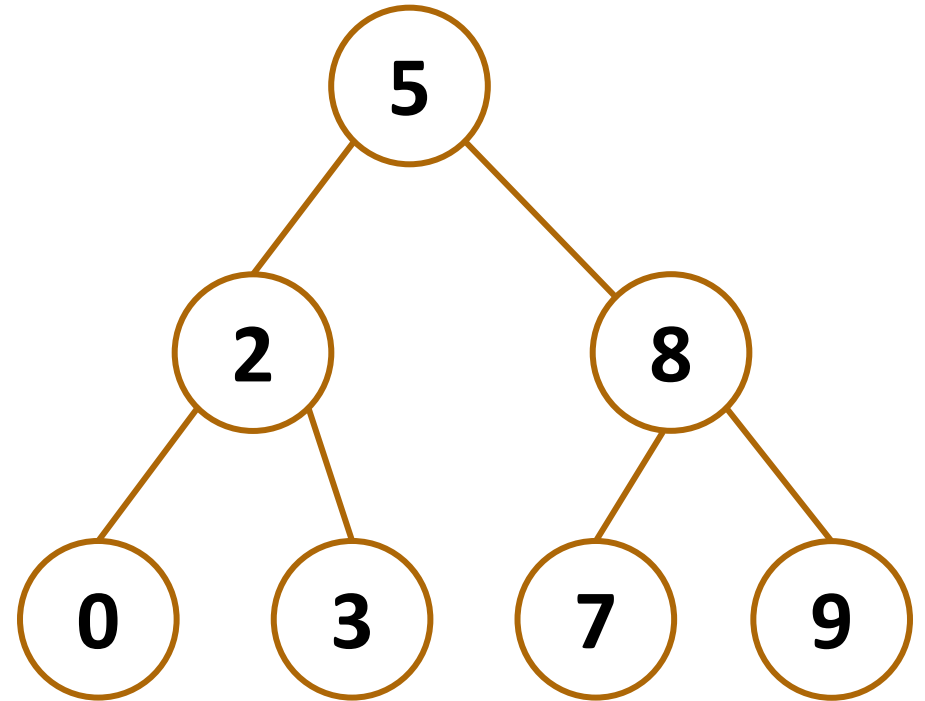


Preorder, inorder, and postorder

Preorder: 5, 2, 0, 3, 8, 7, 9

Inorder: 0, 2, 3, 5, 7, 8, 9

Postorder: 0, 3, 2, 7, 9, 8, 5

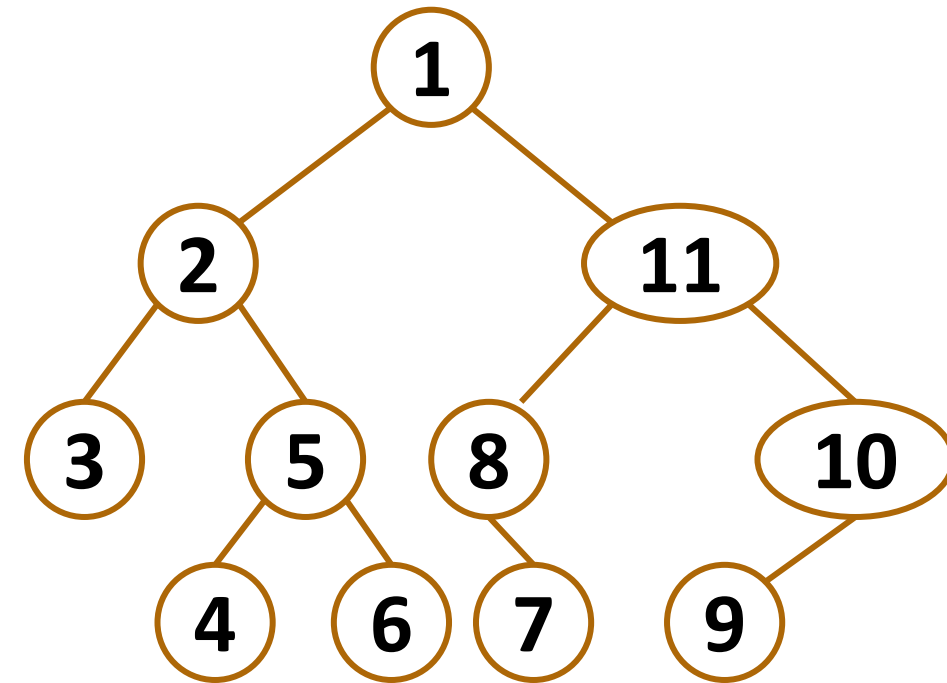


Preorder, inorder, and postorder

Preorder:

Inorder:

Postorder:

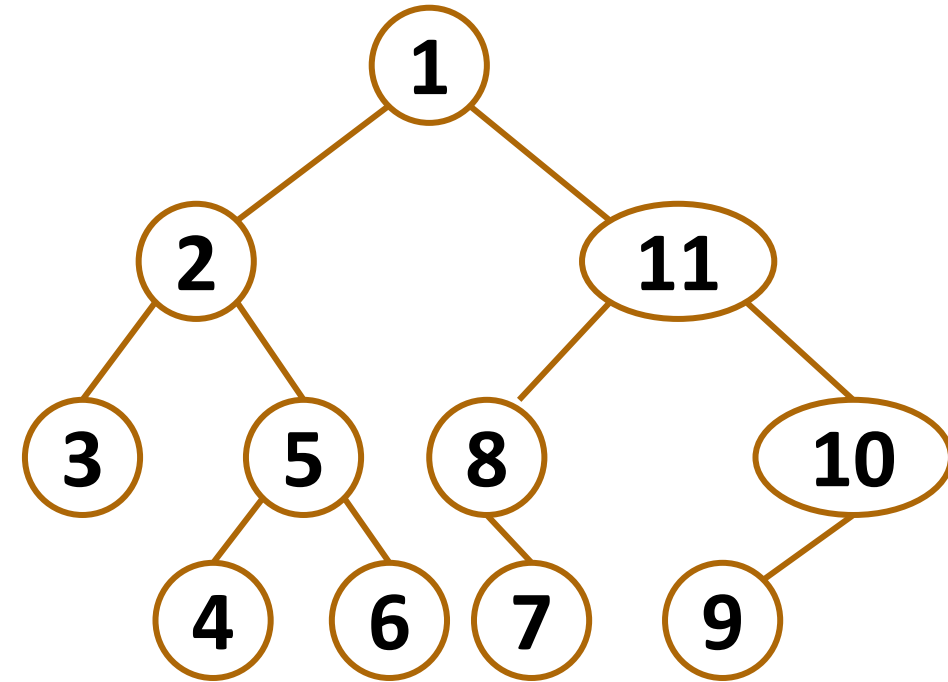


Preorder, inorder, and postorder

Preorder: 1, 2, 3, 5, 4, 6, 11, 8, 7, 10, 9

Inorder: 3, 2, 4, 5, 6, 1, 8, 7, 11, 9, 10

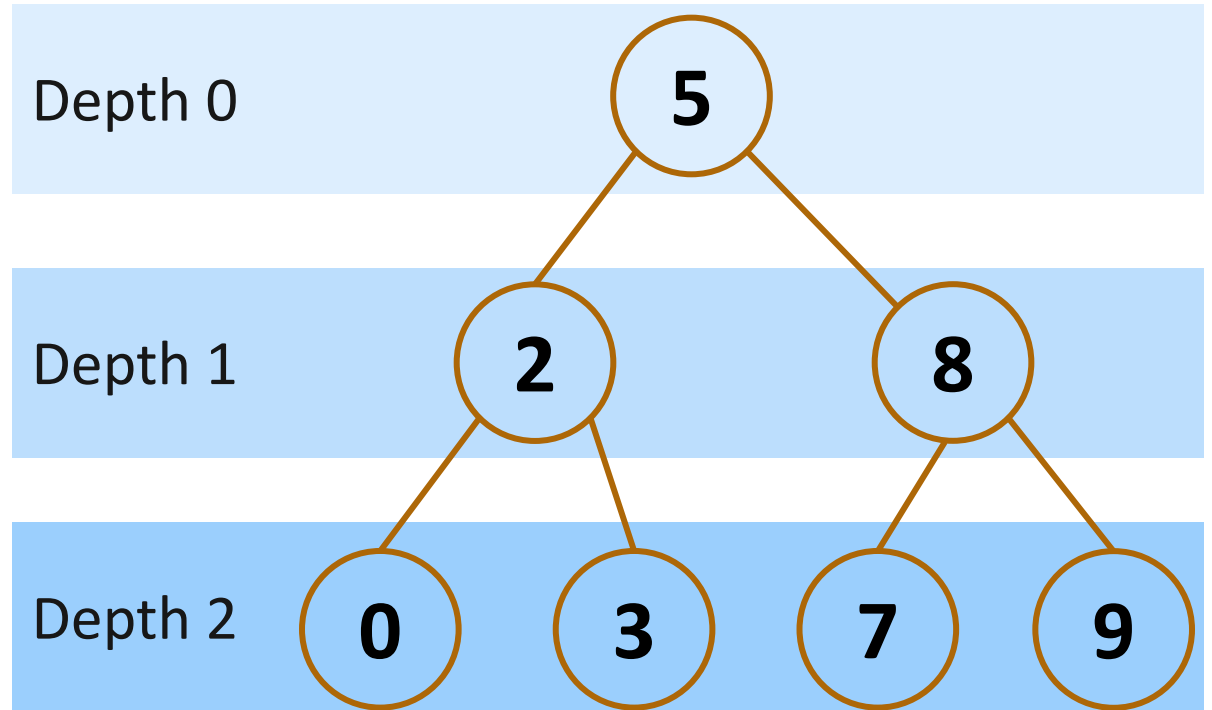
Postorder: 3, 4, 6, 5, 2, 7, 8, 9, 10, 11, 1



BFS: visit the tree level by level

BFS explores all nodes at depth 0, then depth 1, then depth 2, and so on.

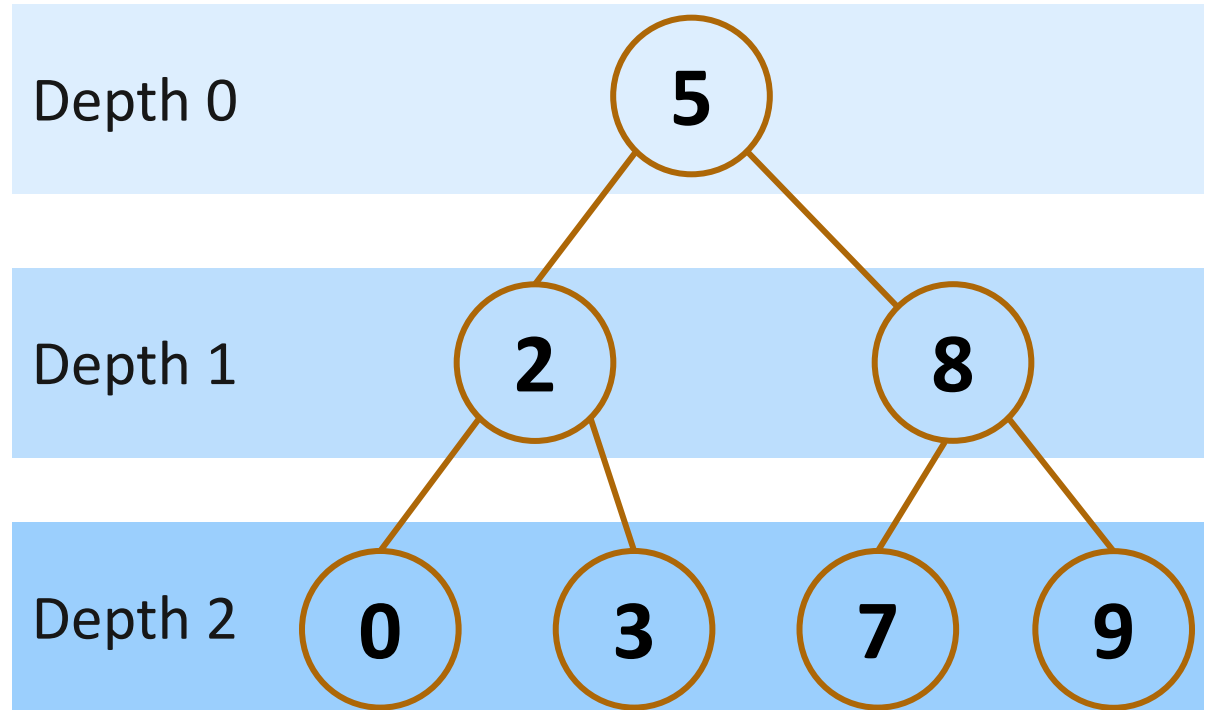
It uses a queue to maintain the current frontier.



BFS: visit the tree level by level

Recursive idea

```
1 Q ← empty queue
2 enqueue(Q, root)
3 while Q ≠ ∅:
4     u ← dequeue(Q)
5     visit(u)
6     for v in children[u]:
7         enqueue(Q, v)
```

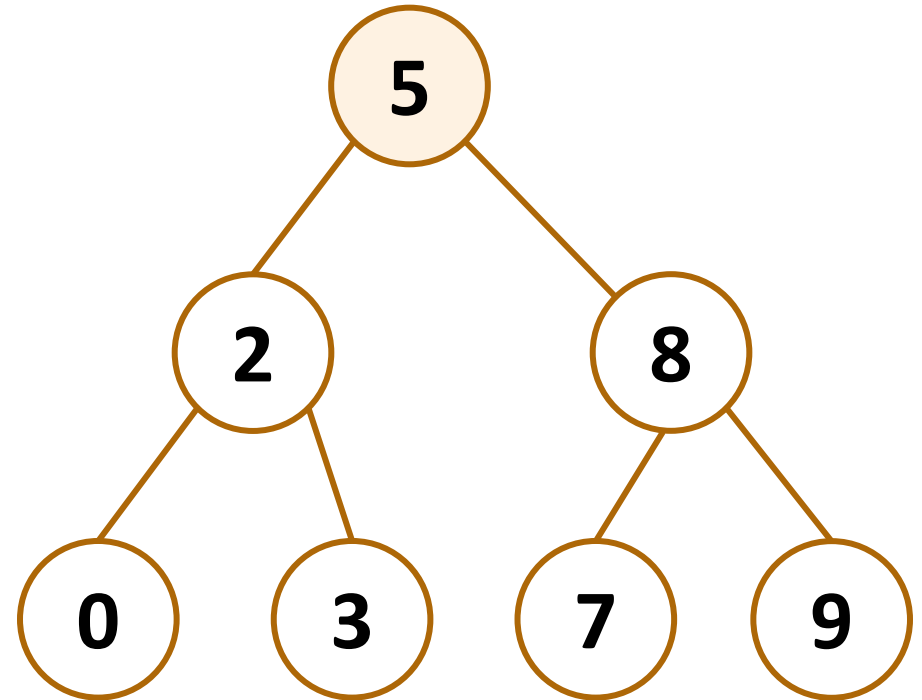


visit u first
then recursively process every child

Visited so far



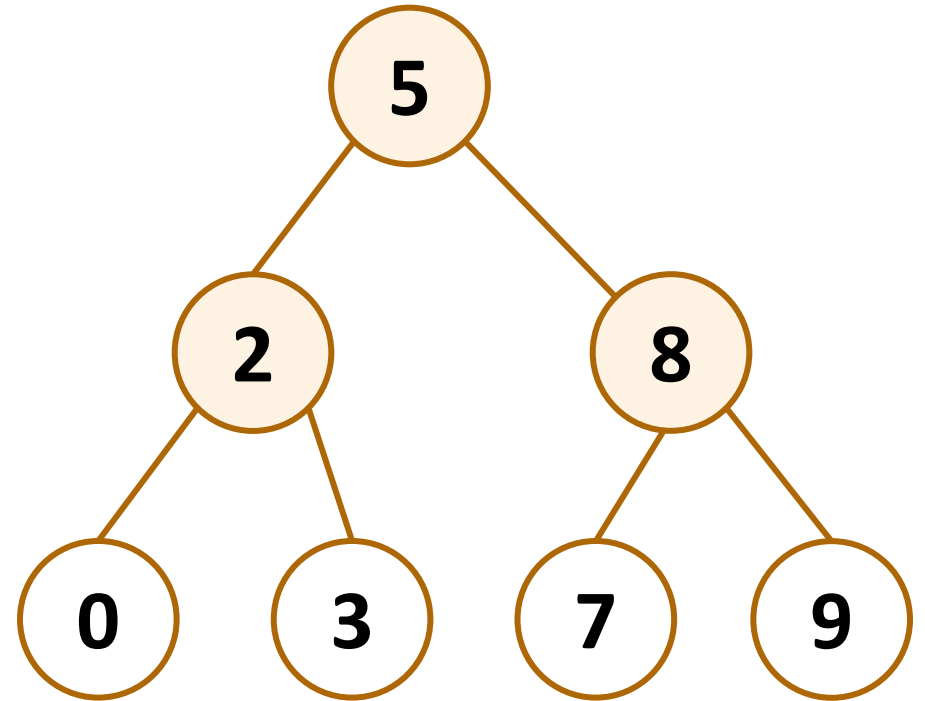
All children of the root join the queue in one step.



Visited so far

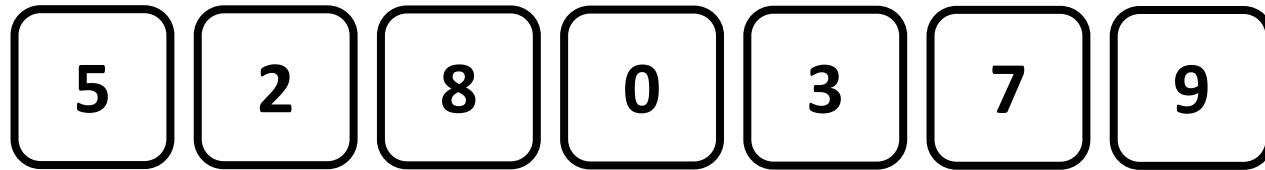


After all depth-1 nodes are processed,
the queue contains depth-2 nodes.

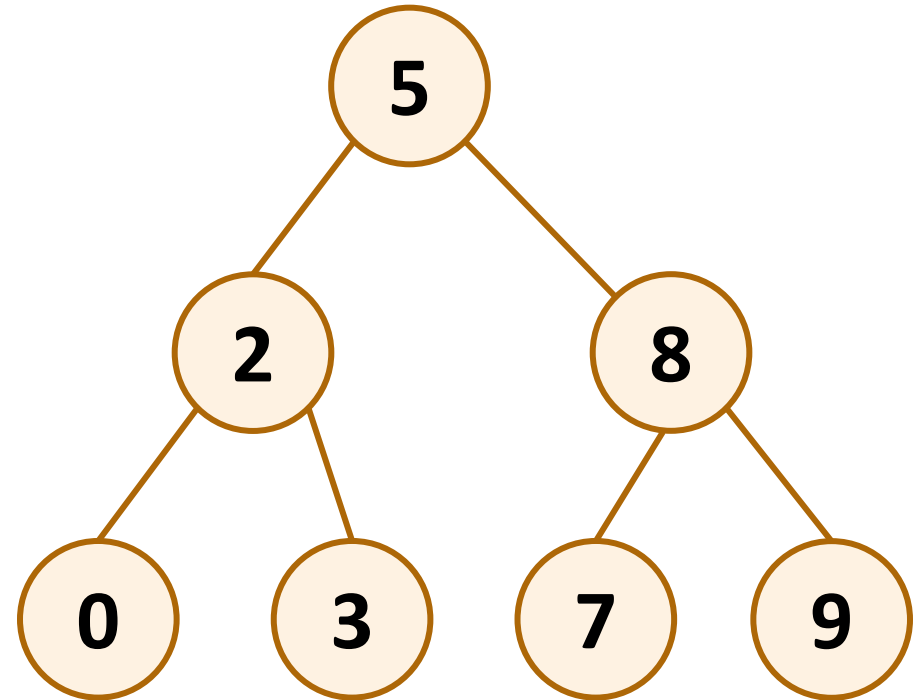


BFS walkthrough

Visited so far



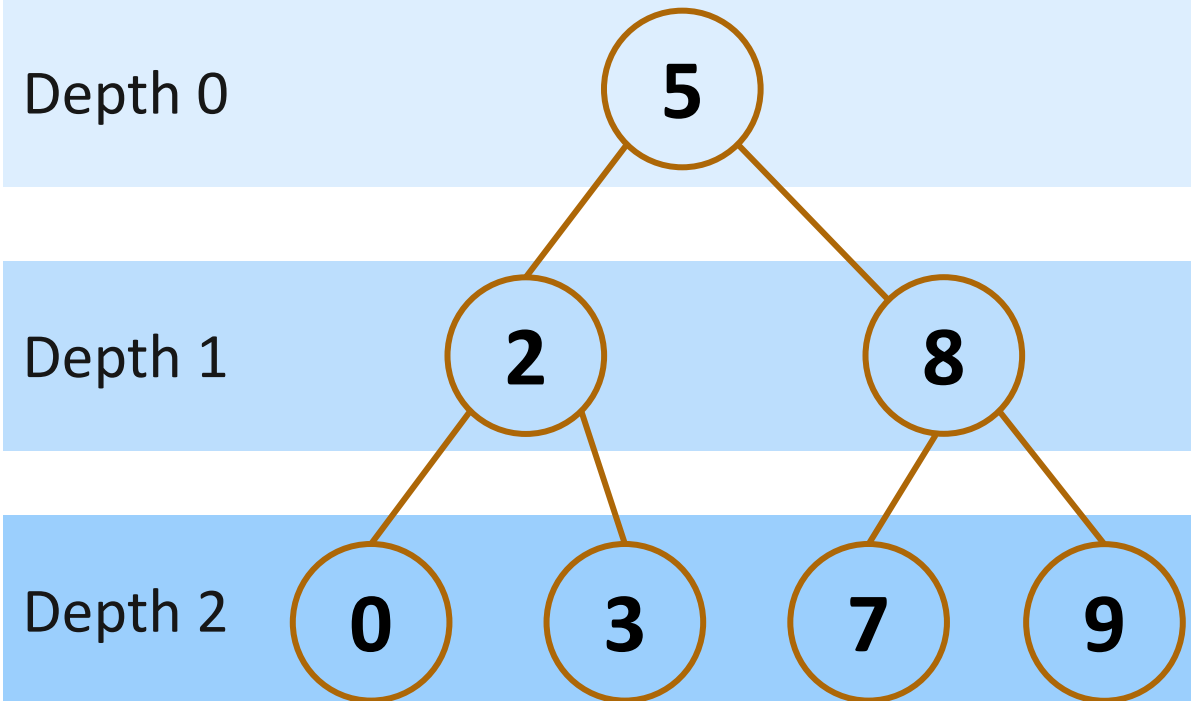
After all depth-1 nodes are processed,
the queue contains depth-2 nodes.



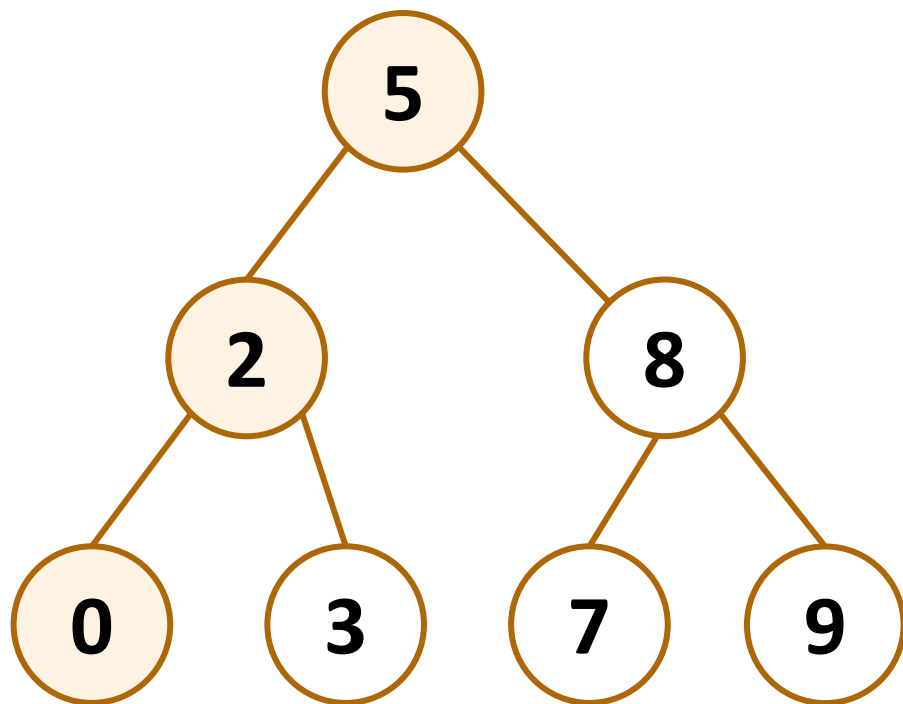
BFS immediately gives node depths

BFS visits nodes level by level, the first time we reach a node we know its depth.

Depth	Nodes
0	5
1	2, 8
2	0, 3, 7, 9

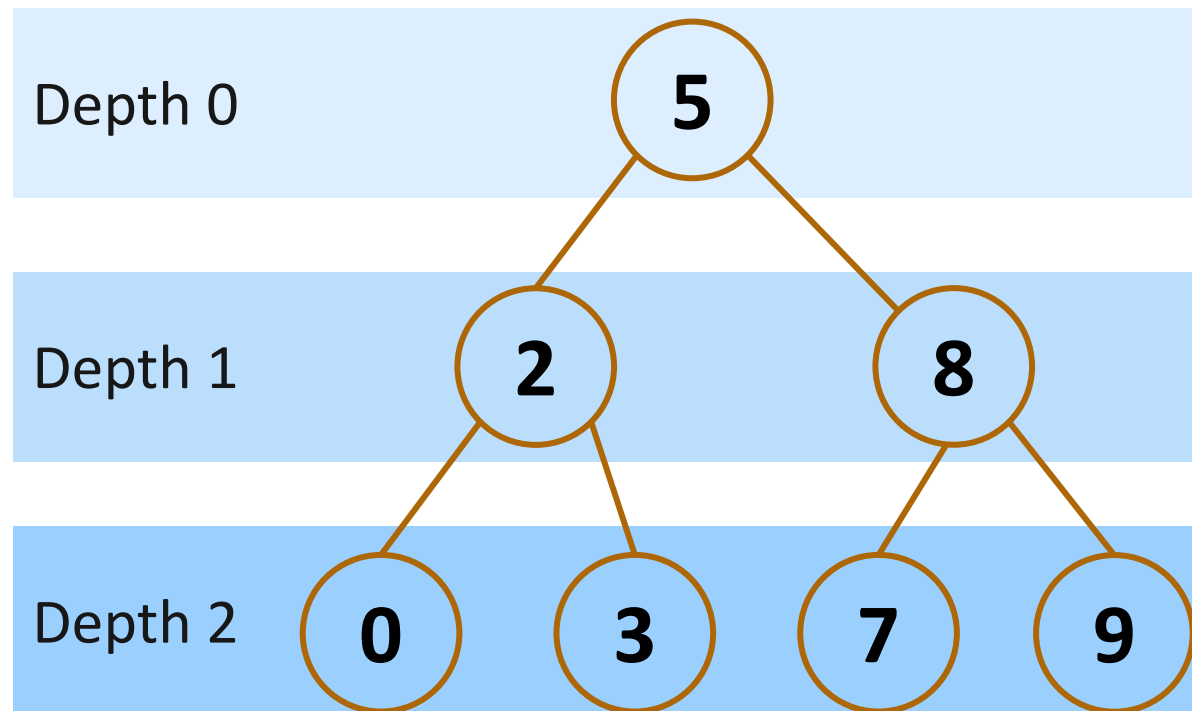


DFS



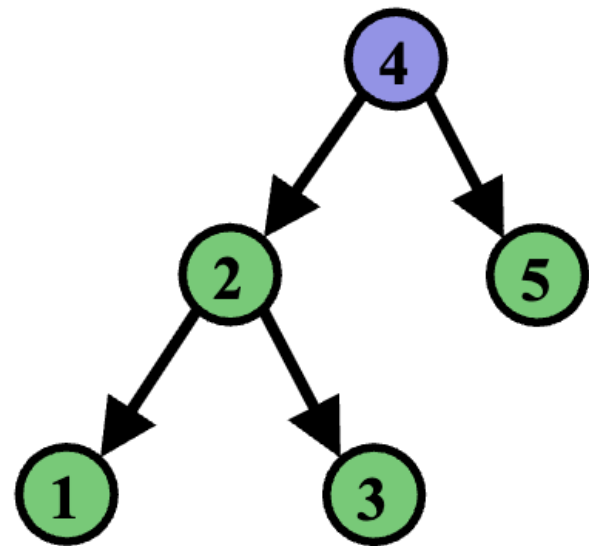
Typical order: 5, 2, 0, 3, 8, 7, 9

BFS

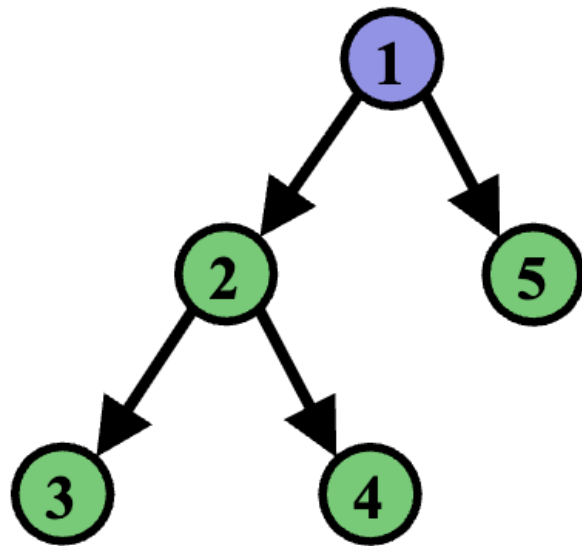


Typical order: 5, 2, 8, 0, 3, 7, 9

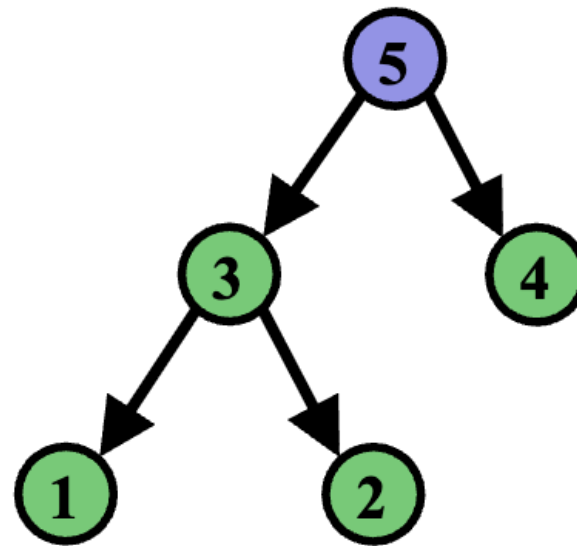
Tree Traversals



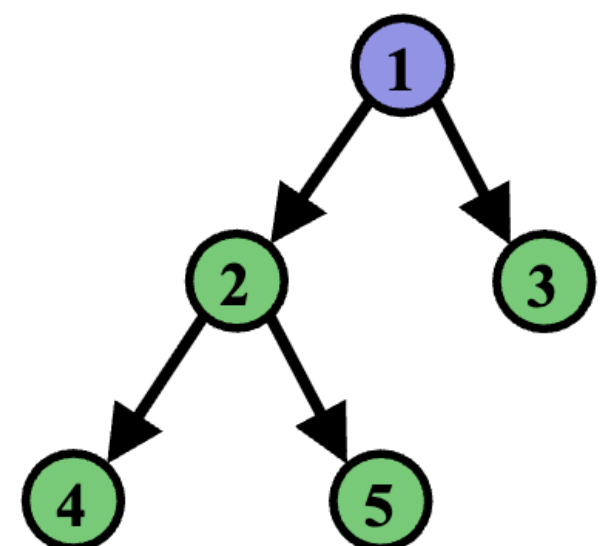
In-order



Pre-order



Post-order

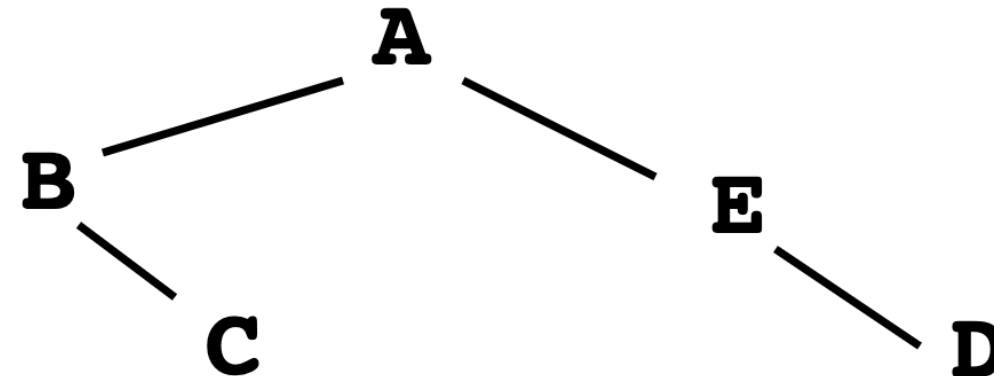
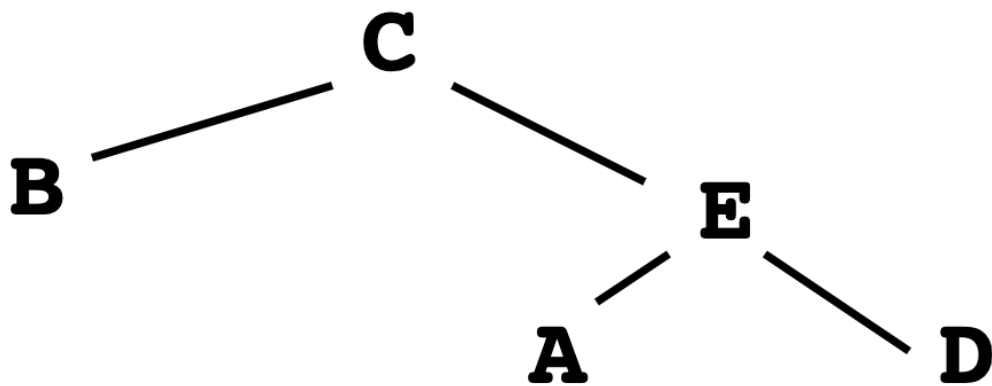


level numbering

Recover tree from traversal

Suppose inorder is B C A E D.
Can we recover the tree uniquely?

No!



Suppose inorder is B C A E D

preorder is A B C D E

Can we determine the tree uniquely? Yes!

- What is root? Preorder tells us: A
- What comes before/after root A? Inorder tells us:
 - Before: B C
 - After: E D
- Now recurse! Figure out left/right subtrees using same technique.

Suppose inorder is B C A E D

preorder is A B C D E

Root is A. left subtree contains B C; right contains E D

Left:

Inorder is B C

Preorder is B C

What is root? Preorder tells us: B

- What comes before/after B?
- Inorder tells us:
 - Before: nothing
 - After: C

Right:

Inorder is E D

Preorder is D E

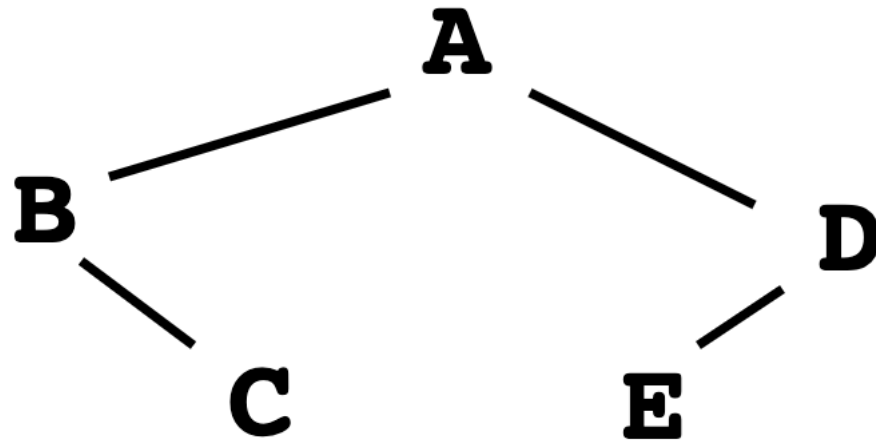
What is root? Preorder tells us: D

- What comes before/after D?
- Inorder tells us:
 - Before: E
 - After: nothing

Recover tree from traversal

Suppose inorder is B C A E D
preorder is A B C D E

Tree is



Recover tree from traversal

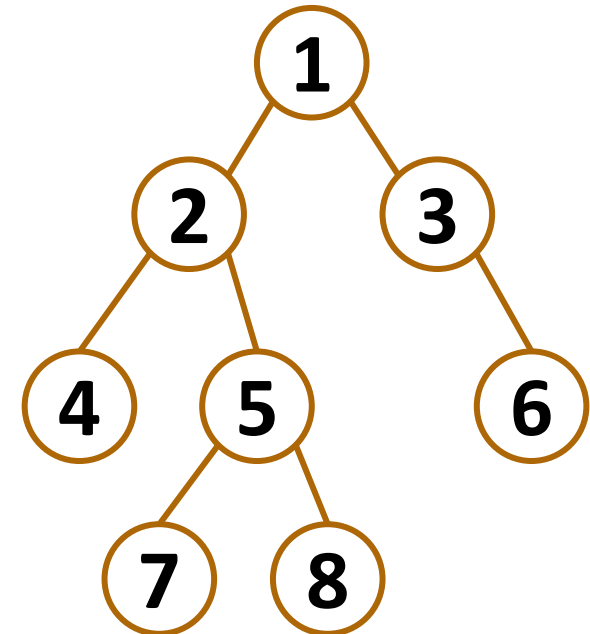
A binary tree can be uniquely determined by

- (pre-order, in-order)
- (post-order, in-order)

Preorder: 1, 2, 4, 5, 7, 8, 3, 6

Inorder: 4, 2, 7, 5, 8, 1, 3, 6

Postorder: 4, 7, 8, 5, 2, 6, 3, 1



Dynamic Programming on Trees

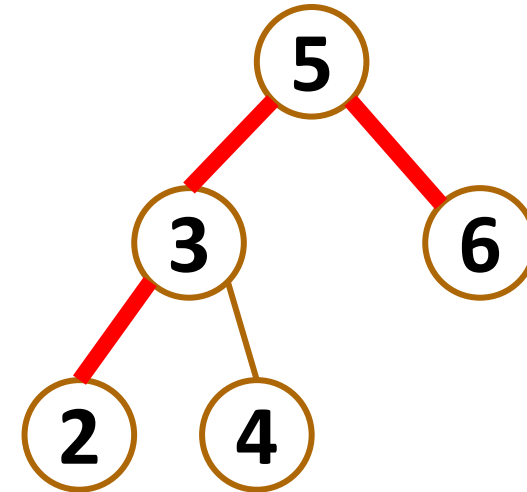
Diameter of a tree

Definition: the length of the longest path between any two nodes of the tree.

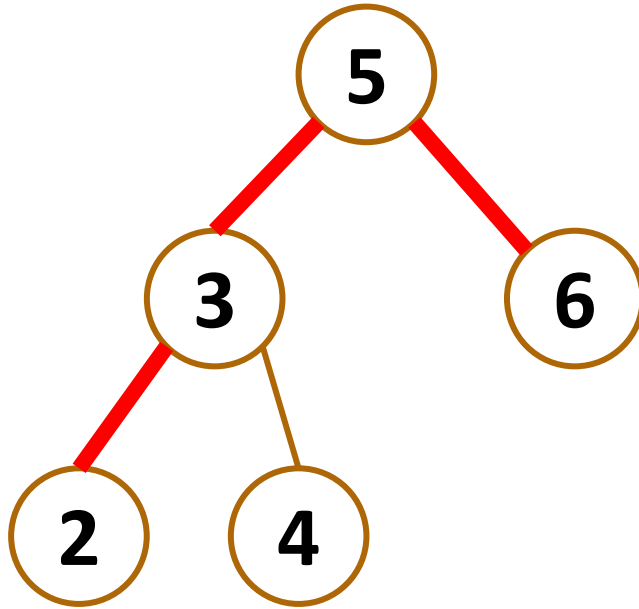
We count edges for the diameter.

- There may be many long paths, but only one maximum length value.

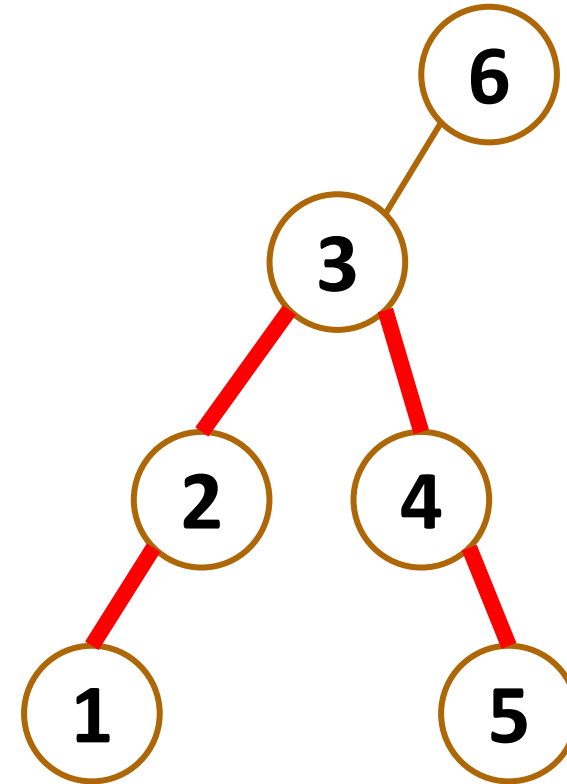
the diameter is 3



Diameter may not pass through the root



diameter is 3



diameter is 4

Run **DFS/BFS** from every node once and take the maximum distance from each run, resulting in an algorithm with complexity $O(N^2)$, where N is the number of nodes.

Step 1: choose **D** as a start

Step 2:

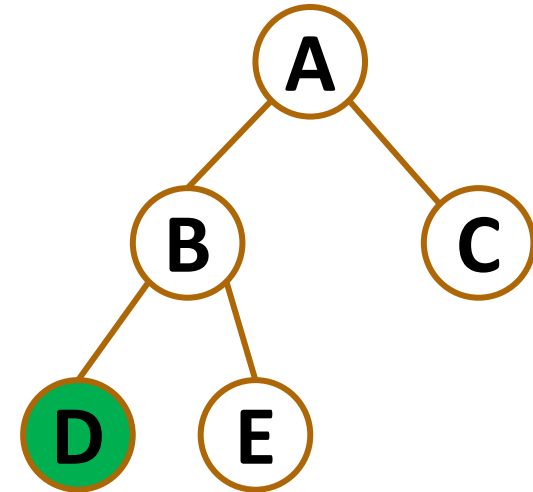
$\text{dist}(D, D) = 0$

$\text{dist}(D, B) = 1$

$\text{dist}(D, E) = 2$

$\text{dist}(D, A) = 2$

$\text{dist}(D, C) = 3$



Run DFS from every node once and take the maximum distance from each run, resulting in an algorithm with complexity $O(N^2)$, where N is the number of nodes.

Step 3: choose **A** as a start

Step 4:

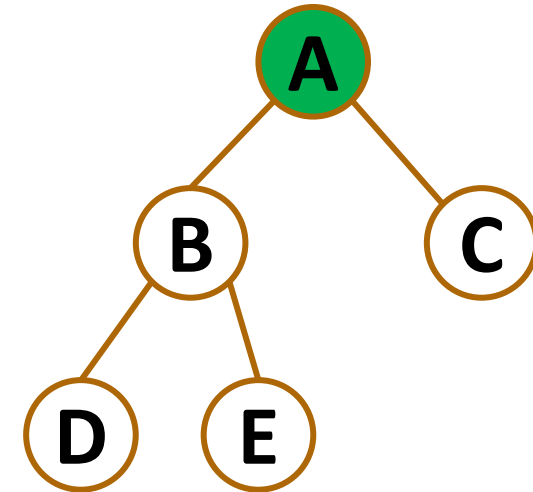
$\text{dist}(A, A) = 0$

$\text{dist}(A, B) = 1$

$\text{dist}(A, C) = 1$

$\text{dist}(A, D) = 2$

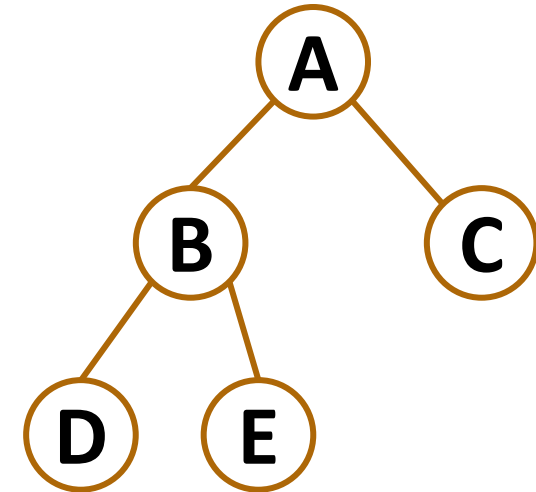
$\text{dist}(A, E) = 2$



Run DFS from every node once and take the maximum distance from each run, resulting in an algorithm with complexity $O(N^2)$, where N is the number of nodes.

Step 5: repeat steps for node B, C, E

Step 6: choose the maximum value across all distances.



The diameter of this tree is 3

We do not have to run a DFS/BFS from every node: we only need to do it **twice**.

- First, we run a DFS/BFS from any node, recording the depth of each node from the starting node.
- Among all the nodes with the maximum depth, choose any of them, and run another DFS/BFS while recording the depth of each node.
- The maximum depth in the second DFS/BFS is the tree diameter.

This results in an algorithm with time complexity $O(N)$, where N is the number of nodes.

- First, we run a DFS/BFS from any node, recording the depth of each node from the starting node.

We randomly choose node **B**:

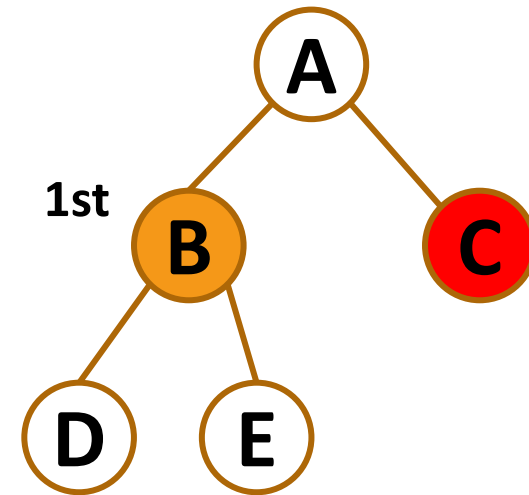
$\text{dist}(B, B) = 0$

$\text{dist}(B, A) = 1$

$\text{dist}(B, D) = 1$

$\text{dist}(B, E) = 1$

$\text{dist}(B, C) = 2$



The maximum depth for node **B** is 2

- Among all the nodes with the maximum depth, choose any of them, and run another DFS/BFS while recording the depth of each node.

We choose node **C**:

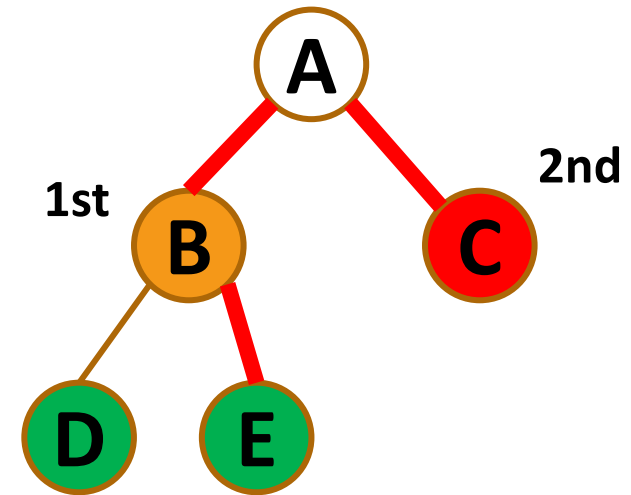
$\text{dist}(C, C) = 0$

$\text{dist}(C, A) = 1$

$\text{dist}(C, B) = 2$

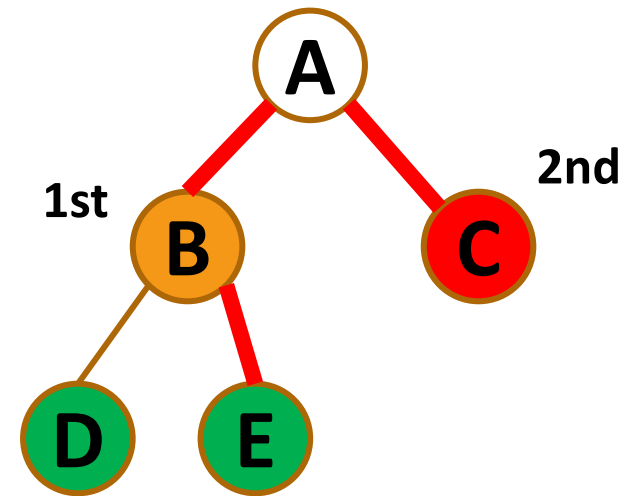
$\text{dist}(C, D) = 3$

$\text{dist}(C, E) = 3$



The maximum depth for node **C** is 3,
and the diameter of this tree is 3

```
Global max_dist = 0
void dfs(int x, int parent, int dist){
    if (dist >= max_dist){
        deepest_node = x;
        max_dist = dist;
    }
    for (auto i : e[x])
        if (i != parent) dfs(i, x, dist + 1);
}
int tree_diameter(){
    max_dist = 0;
    dfs(1, -1, 0);
    max_dist = 0;
    dfs(deepest_node, -1, 0);
    return max_dist;
}
```



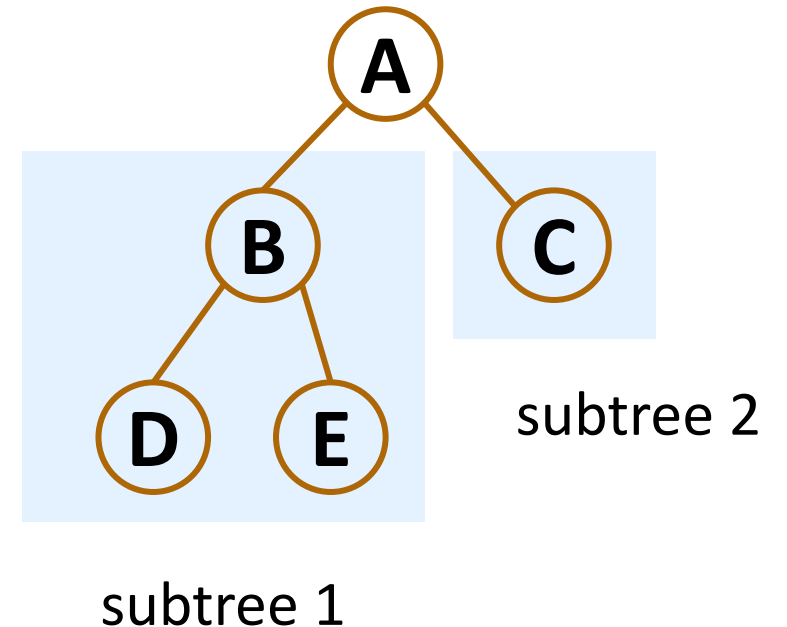
Why does it work?

Key observation for diameter

At a node u , the best path is one of two types:

- It is entirely contained inside one child subtree.
- Or it passes through u and uses the two deepest child branches.

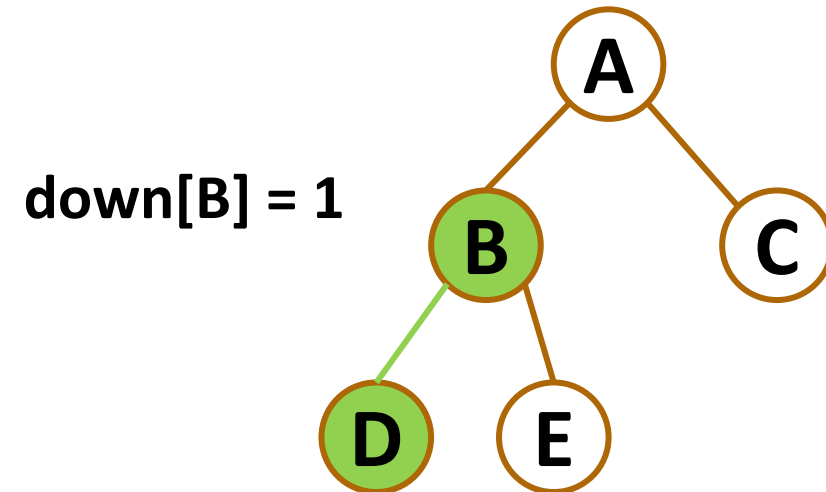
That suggests we should keep the **best** downward path from every node.



Tree DP

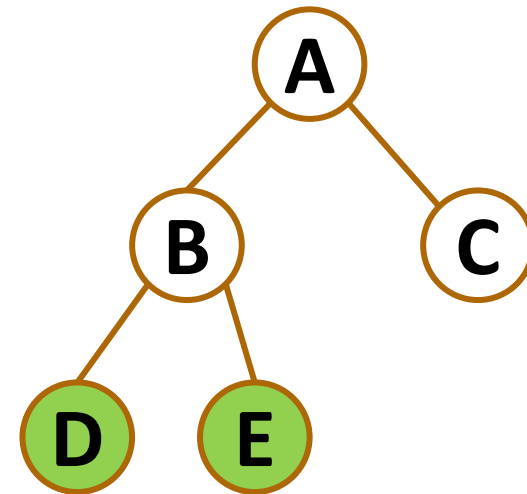
- Assume a tree is rooted
- Use nodes as DP states
- Use parent-children to define transition formula

down[u] = the length of the longest path that starts at u and goes downward into its subtree.



- Leaf: $\text{down}[u] = 0$

No edge can be taken downward.



Suppose u has children $v_1, v_2, \dots, v_k$

For each child v , if we go from u to v , then continue along the best downward path of v , the length becomes:

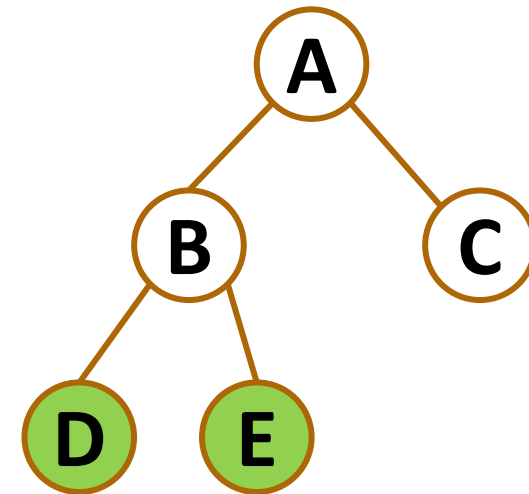
$$\text{down}[v] + 1$$

Let

best1 = the largest value among all $\text{down}[v] + 1$

best2 = the second largest value among all $\text{down}[v] + 1$

(best1 and best2 are associated with different v_i)



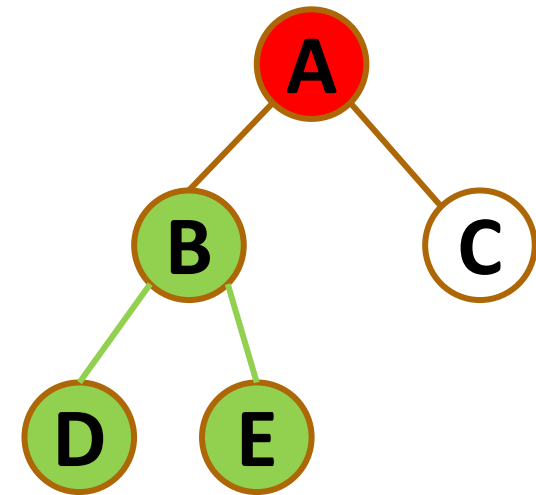
Tree DP - Transition Formula

Longest downward path from u
 $\text{down}[u] = \text{best1}$

Longest path passing through u
 $\text{through}[u] = \text{best1} + \text{best2}$

The diameter is the maximum such value over all nodes:

$\text{diameter} = \max \text{through}[u]$



$\text{through}[B] = 1 + 1 = 2$

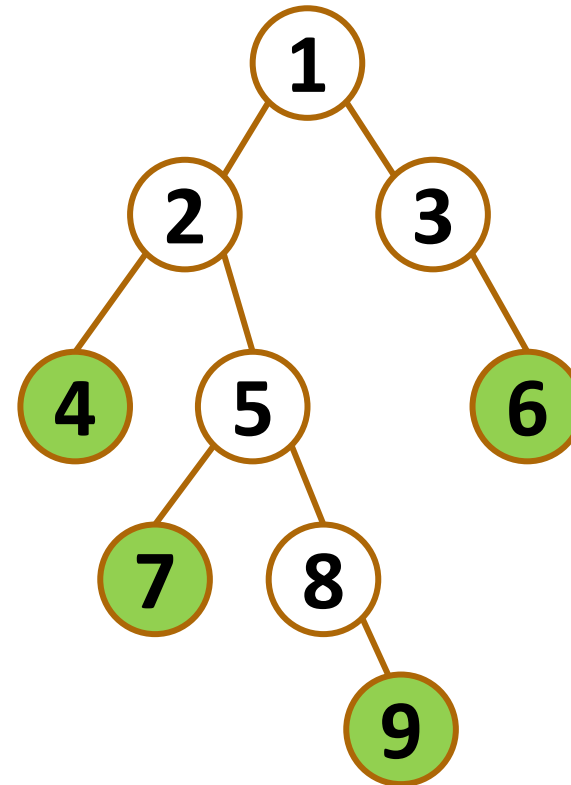
$\text{through}[A] = 2 + 1 = 3$

$\text{diameter} = \text{through}[A] = 3$

Tree DP - another example

For every leaf u ,
 $\text{down}[u] = 0$

$\text{down}[4] = \text{down}[7] = \text{down}[9] =$
 $\text{down}[6] = 0$



Tree DP - another example

$$\text{down}[3] = 1 + \text{down}[6] = 1$$

$$\text{best1} = 1, \text{best2} = 0$$

$$\text{down}[8] = 1 + \text{down}[9] = 1$$

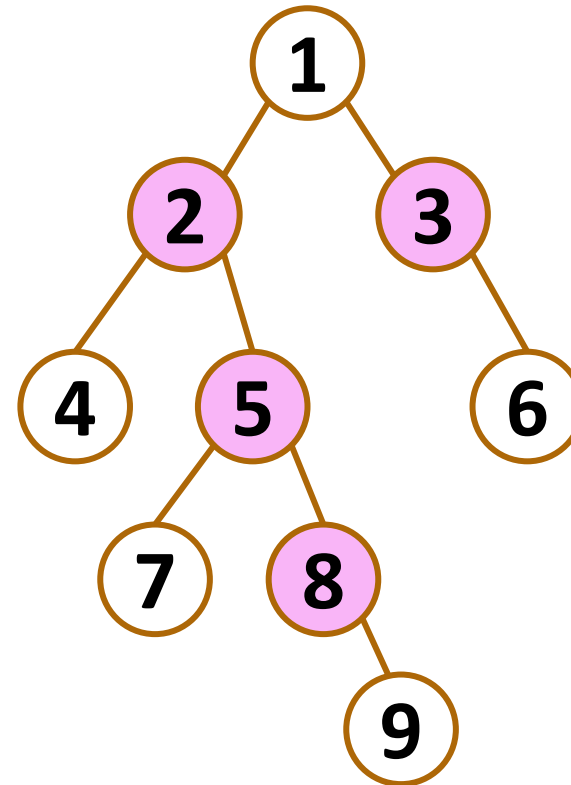
$$\text{best1} = 1, \text{best2} = 0$$

$$\begin{aligned} \text{down}[5] &= 1 + \max(\text{down}[7] + \\ &\quad \text{down}[8]) = 1 + \max(0, 1) = 2 \end{aligned}$$

$$\text{best1} = 2, \text{best2} = 1$$

$$\begin{aligned} \text{down}[2] &= 1 + \max(\text{down}[4] + \\ &\quad \text{down}[5]) = 3 \end{aligned}$$

$$\text{best1} = 3, \text{best2} = 1$$

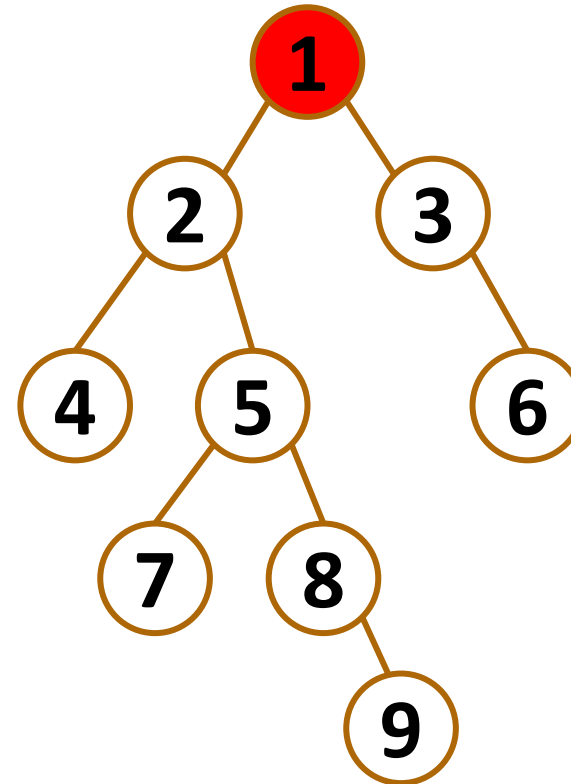


Tree DP - another example

$\text{down}[1] = 1 + \max(\text{down}[2] + \text{down}[3]) = 4$

$\text{best1} = 4, \text{best2} = 2$

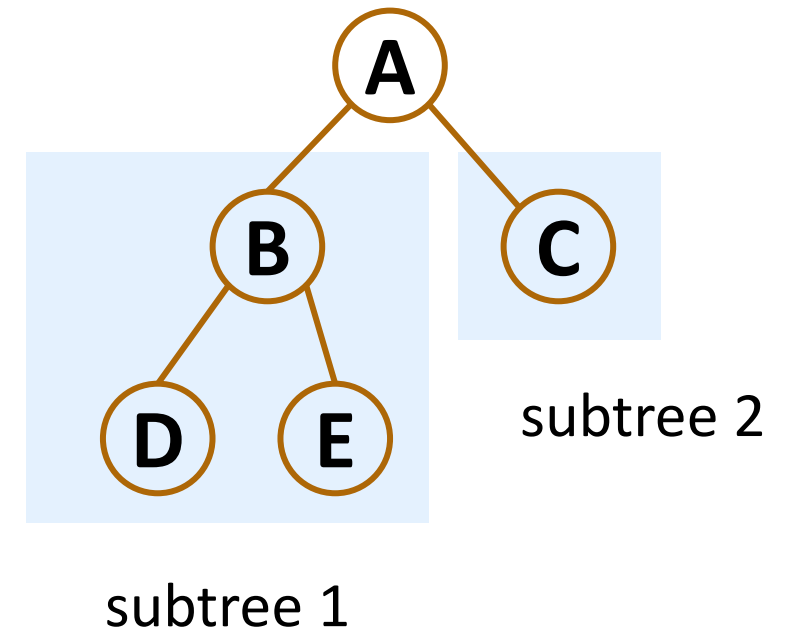
the diameter = $\text{best1} + \text{best2} = 6$



```
1 ans = 0
2
3 def dfs(u, parent):
4     global ans
5     best1 = best2 = 0
6     for v in adj[u]:
7         if v == parent: continue
8         down_v = 1 + dfs(v, u)
9         if down_v > best1:
10             best2 = best1
11             best1 = down_v
12         elif down_v > best2:
13             best2 = down_v
14     ans = max(ans, best1 + best2)
```

- **Time complexity: $O(N)$**

- Given a rooted tree of size N
- Calculate the size of each subtree
- $dp[\text{leaf}] = 1$
- $dp[u] = \text{size of subtree } u$
- $dp[u] = 1 + \sum(dp[v])$ where v is u 's children
- Each node is visited once only
- Time complexity = $O(N)$

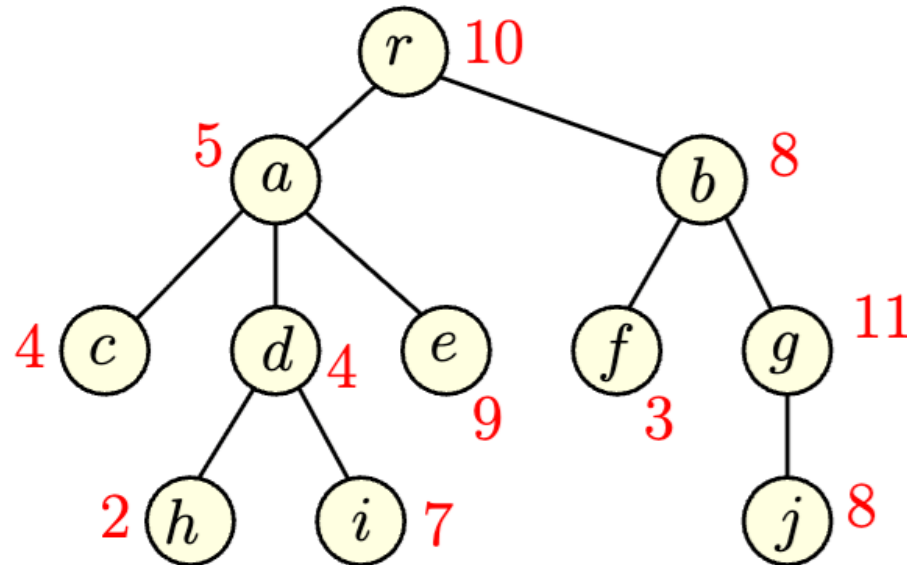


```
Algorithm ComputeSize(u)
  size[u] = 1
  for each child v of u do
    ComputeSize(v)
    size[u] = size[u] + size[v]
  return size[u]
```

Maximum Weight Independent Set in a Tree

Input Tree $T = (V, E)$ and weights $w(v) \geq 0$ for each $v \in V$

Goal Find maximum weight independent set in T



Note: We've studied this part in dynamic programming class.

- **Step 1:** Identify optimal substructure.
- **Step 2:** Find a recursive formulation for the value of the optimal solution
- **Step 3:** Use dynamic programming to find the value of the optimal solution
- **Step 4:** If needed, keep track of some additional info so that the algorithm from Step 3 can find the actual solution.
- **Step 5:** If needed, code this up like a reasonable person.

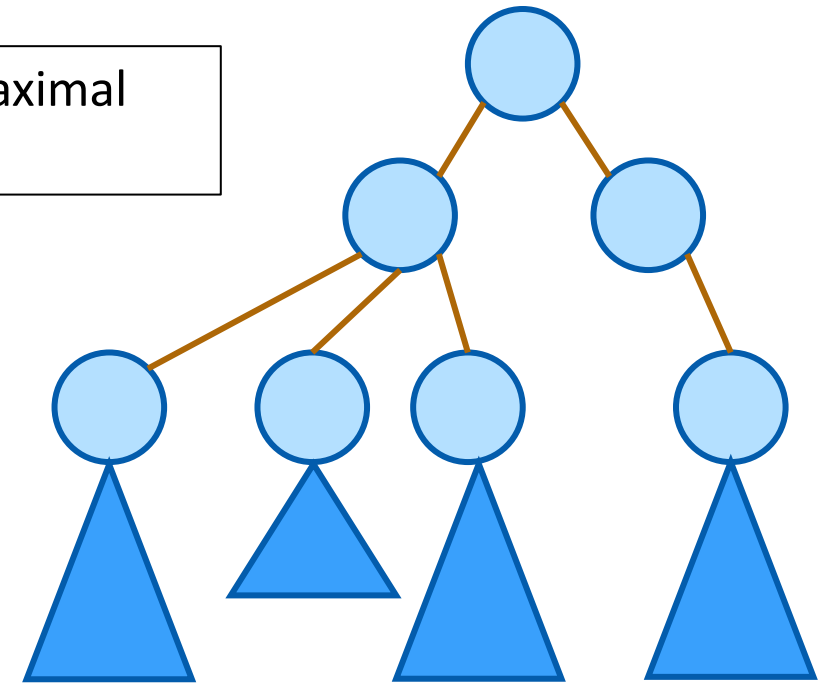
DP Recipe - Step 1

- Natural candidate for v_n is root r of T ? Let O be an optimum solution to the whole problem.

The root is **not** in a maximal independent set.

- There are two cases:

- Case $r \notin O$** : Then O contains an optimum solution $\not\subseteq$ for each subtree of T hanging at a child of r .
- Case $r \in O$** : None of the children of r can be in O . $O - \{r\}$ contains an optimum solution for each subtree of T hanging at a grandchild of r .

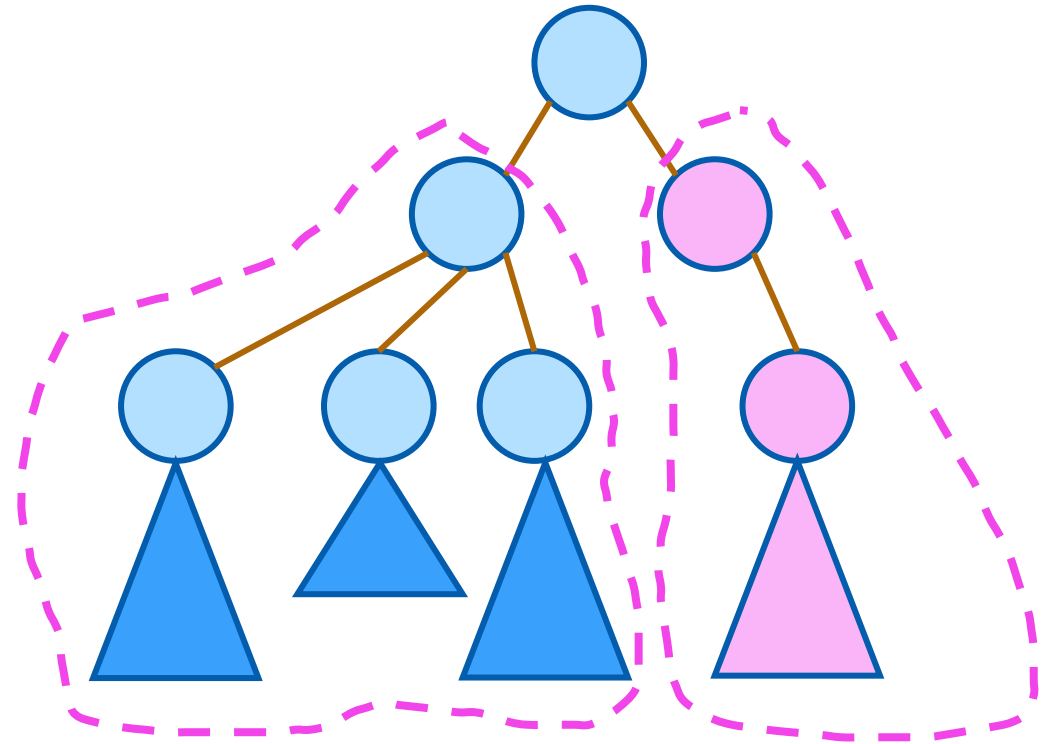


The root is in a maximal independent set.

DP Recipe - Step 1

- Case $r \notin O$: Then O contains an optimum solution for each subtree of T hanging at a child of r .

The root is **not** in a maximal independent set.

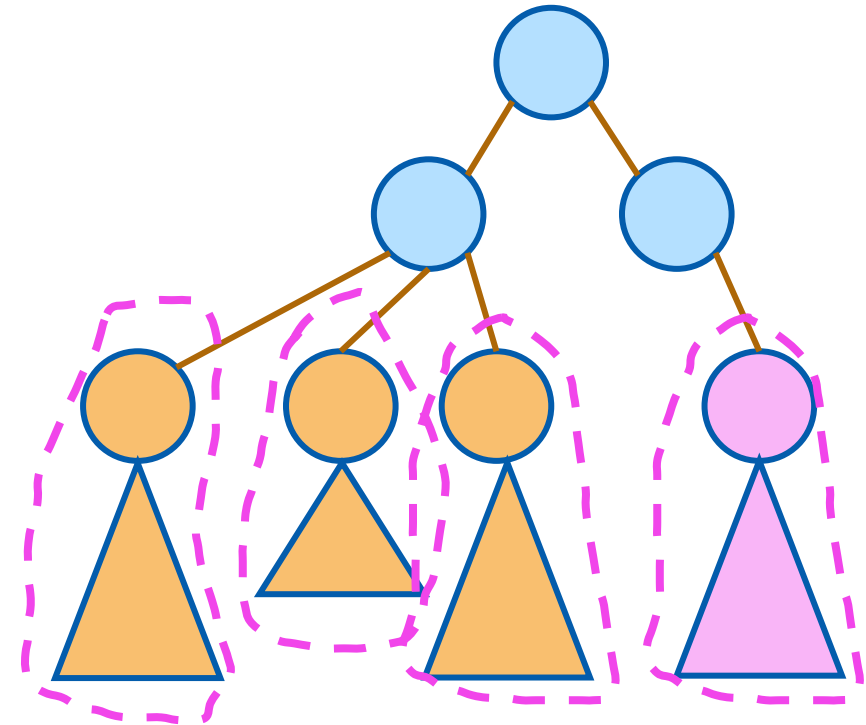


smaller subproblems

DP Recipe - Step 1

- **Case $r \in O$** : None of the children of r can be in O . $O - \{r\}$ contains an optimum solution for each subtree of T hanging at a grandchild of r

The root is in a maximal independent set.



smaller subproblems

DP Recipe - Step 2

$T(u)$: subtree of T hanging at node u

$OPT(u)$: max weighted independent set value in $T(u)$

$$OPT(u) = \max \begin{cases} \sum_{v \text{ child of } u} OPT(v), \\ w(u) + \sum_{v \text{ grandchild of } u} OPT(v) \end{cases}$$

